JobKit - 500 interview and ATS questions

Combined bank: Teradata DBA, Tosca/SAP, Playwright, SQL, and application forms. Each answer is five pointers with working code. Download each type on its own link at https://raviacn95.github.io/job-apply-status/download.html

1. What is notice period?

1. Definition you should say first: Time you must serve after resigning before the last working day. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.
3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.
4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

2. What is current CTC?

1. Definition you should say first: Total annual cost to company you earn now, usually in LPA in India. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.
3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.
4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

3. What is expected CTC?

1. Definition you should say first: Compensation you are targeting for the new role. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.
3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.
4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

4. Are you willing to relocate?

1. Definition you should say first: Say yes only if you can actually move; JobKit default for these profiles is Yes. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.
3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.
4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

5. Do you need visa sponsorship?

1. Definition you should say first: For India-based roles the usual answer is No. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

6. Are you authorized to work in India?

1. Definition you should say first: Yes if you are an Indian citizen or otherwise legally allowed to work. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

7. How did you hear about this job?

1. Definition you should say first: Company careers site, job board, or recruiter post - be truthful. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

8. Why are you leaving?

1. Definition you should say first: Look for a closer skill match, scale, or domain - never badmouth the employer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

9. When can you join?

1. Definition you should say first: Tie it to notice period, for example 30 days after offer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.

3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.

4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

10. Will you work hybrid?

1. Definition you should say first: Yes if the JD allows; confirm location days before you accept. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

11. Are you open to night shifts?

1. Definition you should say first: Only say yes if you truly can; on-call DBA work is different from BPO shifts. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

12. What is your serving notice?

1. Definition you should say first: Same as notice period if you have already resigned; else your contractual notice. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.

3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.

4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

13. Can you join immediately?

1. Definition you should say first: Only if you are already serving notice or are on the bench. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.

3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.

4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

14. Do you have a passport?

1. Definition you should say first: Answer from fact; many banks ask this for background checks. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

15. Have you been through a background check?

1. Definition you should say first: Yes if a prior employer ran one; say so plainly. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

16. Any gap in employment?

1. Definition you should say first: Explain months honestly; do not hide them. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

17. Are you bound by a non-compete?

1. Definition you should say first: Follow the contract you signed; most India IT bonds are limited. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

18. Can you provide payslips?

1. Definition you should say first: Yes if you have them; never fake documents. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

19. What is variable pay?

1. Definition you should say first: The bonus or incentive part of CTC that is not guaranteed. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.
3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.
4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

20. What is take-home vs CTC?

1. Definition you should say first: Take-home is in-hand after tax and deductions; CTC is the full employer cost. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.
3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.
4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

21. Do you have PF and UAN?

1. Definition you should say first: Yes if you were on payroll; share UAN only on official HR forms. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

22. Are you comfortable with a bond?

1. Definition you should say first: Decide before you apply; do not agree in a form if you will refuse later. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

23. Current location?

1. Definition you should say first: City you live in now, for example Bengaluru. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.
3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

24. Preferred location?

1. Definition you should say first: Where you will work; match the JD. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

25. Highest education?

1. Definition you should say first: Degree, school, and year from your profile. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

26. Current employer?

1. Definition you should say first: Your latest company name. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

27. Current designation?

1. Definition you should say first: Your latest title. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
```

```
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

28. Total years of experience?

1. Definition you should say first: Full-time years from your profile, not internships unless asked. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

29. Relevant years of experience?

1. Definition you should say first: Years on the track in the JD, for example Teradata or Tosca. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

30. LinkedIn URL?

1. Definition you should say first: Your public profile URL. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

31. Cover letter in one line?

1. Definition you should say first: I am applying for this role because my production work matches the JD skills. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

32. Why this company?

1. Definition you should say first: Point to the product, scale, or domain in the JD, not generic praise. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

33. Why this role?

1. Definition you should say first: Map two or three JD skills to work you have actually done. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

34. What is your strength?

1. Definition you should say first: A skill from the JD you can prove with a metric. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

35. What is your weakness?

1. Definition you should say first: A real gap you are closing, not a humble-brag. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

36. Tell me about yourself.

1. Definition you should say first: 60-second story: years, stack, one result, why this role. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

37. Describe a conflict at work.

1. Definition you should say first: Situation, what you did, outcome, no blame. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

38. Describe a production incident.

1. Definition you should say first: Impact, what you checked, fix, and how you prevented a repeat. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

39. Are you a team player?

1. Definition you should say first: Give one example of pairing or handover, not a slogan. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

40. How do you handle pressure?

1. Definition you should say first: Prioritize P1, communicate ETA, and write the timeline after. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

41. Where do you see yourself in 5 years?

1. Definition you should say first: Deeper in the same craft (lead DBA or automation architect), not a random MBA line. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

42. Do you have managerial experience?

1. Definition you should say first: Only if you have led people; else say you have led workstreams. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. These are story questions. Map JD skills to work you did. Never badmouth. Weakness is a real gap you are closing.

3. 60-second pitch: years, stack, one metric, why this role. Incident: impact, checks, fix, prevention.

4. Working code / syntax (write this on ATS form values):

```
Pitch: 8 years Teradata DBA. Cut a 47-min batch to 29 by fixing PI skew and stats on fact_orders. Applying because this JD owns BAR and TASM on Vantage.
Weakness: Fiori exposure is limited; I am ramping HTML engine tests on the QA Fiori launchpad.
```

5. A strength that is not on the JD ('I am a perfectionist') scores zero.

43. Salary negotiation opening?

1. Definition you should say first: State expected CTC from the profile and wait; do not undercut 30 LPA search floor. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

44. Counter-offer from current company?

1. Definition you should say first: Decide before the call; most people who stay after a counter leave within a year. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. ATS forms store legal facts. Treat every field as a document you may have to defend in HR and BGV.

3. Answer from your profile only. If the form is unclear, put the conservative true value and explain in interview.

4. Working code / syntax (write this on ATS form values):

```
Notice period: 30 days
Current CTC: 28 LPA
Expected CTC: 32 LPA
Willing to relocate: Yes
Visa sponsorship: No
```

5. Never invent CTC, notice, or tools. A mismatch with payslips or UAN is an auto-reject.

45. Can you work from office 5 days?

1. Definition you should say first: Answer the real policy; do not say yes and then refuse. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

46. Do you have a laptop?

1. Definition you should say first: Yes if you use your own; company kit is usual after joining. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

47. Any ongoing interviews?

1. Definition you should say first: You may say you are exploring options without naming companies. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

48. Are you interviewing with our competitor?

1. Definition you should say first: Do not volunteer names; stay professional. Then spend the rest of the answer on

architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

49. Can we contact your current manager?

1. Definition you should say first: Usually after resignation; say so. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

50. References?

1. Definition you should say first: Keep two ready: a lead and a peer who know your work. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

51. What is garden leave?

1. Definition you should say first: Paid period after resigning when you do not work, if the contract says so. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.

3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.

4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

52. What is a buyout?

1. Definition you should say first: Paying remaining notice so you can join earlier, if both employers agree. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.

3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.

4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

53. What is serving vs remaining notice?

1. Definition you should say first: Serving means you already resigned; remaining is days left. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.
3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.
4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

54. Are you on a contract?

1. Definition you should say first: Say payroll vs vendor vs contract-to-hire exactly. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

55. Open to contract roles?

1. Definition you should say first: Only if you want them; many 30 LPA+ hunts are full-time. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

56. Open to startup vs MNC?

1. Definition you should say first: Pick from the JD; do not apply to both with opposite stories. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

57. Night on-call for DBA?

1. Definition you should say first: Production DBA work often includes on-call; confirm the roster. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.
3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.
4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
```

```
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

58. Weekend releases?

1. Definition you should say first: Common in banks; say yes only if you have done them. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

59. Travel percentage?

1. Definition you should say first: Answer from fact; many India roles are local plus rare onsite. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Location and shift answers must match how you will actually work. DBA on-call is not the same as BPO nights.

3. If the JD is Bengaluru hybrid 3 days, do not mark 'remote only' unless you will refuse the offer.

4. Working code / syntax (write this on ATS form values):

```
Current city: Bengaluru
Preferred: Bengaluru
Willing to relocate: Yes
Visa sponsorship: No
Work authorization India: Yes
On-call: Yes for production DBA roster
```

5. Saying yes to 5-day office and then demanding remote after offer is a withdrawn offer.

60. Language requirements?

1. Definition you should say first: English plus any language you actually speak. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

61. Disability or accommodations?

1. Definition you should say first: Share only what you want HR to know; it is optional. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

62. Relatives in the company?

1. Definition you should say first: Disclose if the form asks; conflicts of interest matter. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

63. Have you applied here before?

1. Definition you should say first: Tell the truth; ATS stores history. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.
3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.
4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

64. How soon for an interview?

1. Definition you should say first: Give two windows in the next week. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.
3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.
4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

65. Online vs onsite interview?

1. Definition you should say first: Follow their process; keep a quiet room for video. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.
3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.
4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

66. What to take to an interview?

1. Definition you should say first: ID, updated resume, and questions for them. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.
3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

67. Questions to ask the interviewer?

1. Definition you should say first: Team size, on-call, tech stack version, and success in 90 days. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

68. How do you learn a new tool?

1. Definition you should say first: Sandbox, vendor docs, then a small production-safe change. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Learn in a sandbox, then a production-safe change. Document in the ticket + runbook. Estimate as range including review and rollback. Unclear JD: ask systems and success metric.

3. Handover = access, jobs, risks, live walkthrough. Not a chat dump on last day.

4. Working code / syntax (write this on ATS form values):

```
Estimate: 1d analysis + 1d change + 0.5d review + 0.5d rollback drill = 3d (range 2.5-4)
Handover checklist: IDs, BAR job name, TASM WD, on-call roster
```

5. A 2-hour estimate for a PI change on a 10 TB table is not credible.

69. How do you document work?

1. Definition you should say first: Runbook plus change ticket; not only chat messages. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Learn in a sandbox, then a production-safe change. Document in the ticket + runbook. Estimate as range including review and rollback. Unclear JD: ask systems and success metric.

3. Handover = access, jobs, risks, live walkthrough. Not a chat dump on last day.

4. Working code / syntax (write this on ATS form values):

```
Estimate: 1d analysis + 1d change + 0.5d review + 0.5d rollback drill = 3d (range 2.5-4)
Handover checklist: IDs, BAR job name, TASM WD, on-call roster
```

5. A 2-hour estimate for a PI change on a 10 TB table is not credible.

70. How do you estimate a task?

1. Definition you should say first: Break it, add review and rollback time, then give a range. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Learn in a sandbox, then a production-safe change. Document in the ticket + runbook. Estimate as range including review and rollback. Unclear JD: ask systems and success metric.

3. Handover = access, jobs, risks, live walkthrough. Not a chat dump on last day.

4. Working code / syntax (write this on ATS form values):

```
Estimate: 1d analysis + 1d change + 0.5d review + 0.5d rollback drill = 3d (range 2.5-4)
Handover checklist: IDs, BAR job name, TASM WD, on-call roster
```

5. A 2-hour estimate for a PI change on a 10 TB table is not credible.

71. What is a good handover?

1. Definition you should say first: Access, runbooks, open risks, and a live walkthrough. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Learn in a sandbox, then a production-safe change. Document in the ticket + runbook. Estimate as range including review and rollback. Unclear JD: ask systems and success metric.

3. Handover = access, jobs, risks, live walkthrough. Not a chat dump on last day.

4. Working code / syntax (write this on ATS form values):

```
Estimate: 1d analysis + 1d change + 0.5d review + 0.5d rollback drill = 3d (range 2.5-4)
Handover checklist: IDs, BAR job name, TASM WD, on-call roster
```

5. A 2-hour estimate for a PI change on a 10 TB table is not credible.

72. How do you deal with an unclear JD?

1. Definition you should say first: Ask which systems are in scope and what success looks like. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Learn in a sandbox, then a production-safe change. Document in the ticket + runbook. Estimate as range including review and rollback. Unclear JD: ask systems and success metric.

3. Handover = access, jobs, risks, live walkthrough. Not a chat dump on last day.

4. Working code / syntax (write this on ATS form values):

```
Estimate: 1d analysis + 1d change + 0.5d review + 0.5d rollback drill = 3d (range 2.5-4)
Handover checklist: IDs, BAR job name, TASM WD, on-call roster
```

5. A 2-hour estimate for a PI change on a 10 TB table is not credible.

73. STAR method?

1. Definition you should say first: Situation, Task, Action, Result - use numbers in Result. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

74. What is a PIP?

1. Definition you should say first: Performance improvement plan; answer only if you have been on one. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

75. Reason for a short stint?

1. Definition you should say first: Project end, location, or scope mismatch - keep it factual. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

76. Can you share client names?

1. Definition you should say first: Only if NDA allows; else describe industry and scale. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

77. NDA in interviews?

1. Definition you should say first: Do not paste internal docs; speak in general terms. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

78. What is double employment?

1. Definition you should say first: Working two full-time jobs at once; most employers forbid it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BGV matches documents. Gaps need dates. PF/UAN from payroll. NDA: no client paste. Double employment is forbidden. PIP: only if asked and true.

3. Keep payslips, Form 16, degree, relieving letter in one folder. Same employer name spelling everywhere.

4. Working code / syntax (write this on ATS form values):

```
Highest education: B.E. 2016 <College>
Total exp: 8 years
Relevant: 6 years Teradata
UAN: share only on HR portal
Client names: 'large Indian bank' if NDA blocks the brand
```

5. Fake experience years are the fastest BGV fail.

79. Notice buyout who pays?

1. Definition you should say first: Sometimes the new employer; get it in the offer letter. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Notice is contractual time after resignation. Serving means you already resigned. Remaining is days left. Buyout is paying the balance. Garden leave is paid absence.

3. Put the number from your letter. If they need earlier, talk buyout after offer, not in the first ATS box.

4. Working code / syntax (write this on ATS form values):

```
Notice period (days): 30
Currently serving notice: No
Earliest join: 30 days after resignation / offer
Buyout: negotiable after written offer
```

5. Saying 'immediate' when your letter says 90 days fails BGV and burns the recruiter.

80. Expected CTC in words?

1. Definition you should say first: Match the number on the form; do not put a range if they want one figure. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

81. Current CTC breakup?

1. Definition you should say first: Have basic, HRA, special, and bonus ready if they ask. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

82. Last hike percentage?

1. Definition you should say first: Use the real number from your letter. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

83. Holding an offer?

1. Definition you should say first: You may say you have one without inflating the number. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

84. Joining bonus expectation?

1. Definition you should say first: Do not demand it unless they offer; it is not guaranteed. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

85. Stock or RSU?

1. Definition you should say first: Ask after base and bonus are clear. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
```

```
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

86. Shift allowance?

1. Definition you should say first: Relevant for 24x7 support roles; confirm in the offer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

87. Relocation bonus?

1. Definition you should say first: Ask if you must move cities. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. CTC is annual employer cost. Take-home is after tax. Variable is not guaranteed. Expected CTC in this kit stays at or above the 30 LPA search floor unless you personally change it.

3. State one expected number. Breakup ready: basic, HRA, special, bonus. Do not inflate.

4. Working code / syntax (write this on ATS form values):

```
Current CTC: 28 LPA (fixed 24 + variable 4)
Expected CTC: 32 LPA
Take-home approx: as per last payslip
RSU/joining bonus: discuss after base is agreed
```

5. Different CTC on two forms for the same company is treated as dishonesty.

88. Probation length?

1. Definition you should say first: Typical 3-6 months; confirm confirmation criteria. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. You may say you are exploring. Do not name competitors. Manager contact after resignation. References: a lead and a peer. STAR is situation-task-action-result with a number.

3. Offer two interview slots this week. Ask them about on-call, version of Teradata/Tosca, and 90-day success.

4. Working code / syntax (write this on ATS form values):

```
STAR: S=month-end spool full T=restore dashboard A=ABORT runaway + collect stats + add date PPI prune R=job 29 min, no freeze miss
Questions: TASM in place? DEX agent count? BAR last restore drill?
```

5. Asking only about WFH and CTC in round 1 signals you did not read the JD.

89. What is Teradata?

1. Definition you should say first: A shared-nothing MPP database for large analytic workloads. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata is a shared-nothing MPP analytic database. Vantage is the current platform branding (SQL plus analytics). Each AMP owns a slice of every table.

3. Draw PE -> BYNET -> AMPs. Mention PI, stats, TASM, and BAR in the first minute.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.DiskSpaceV WHERE DatabaseName = 'EDW';
HELP SESSION;
SELECT HASHAMP() AS amps;
```

5. Calling it 'just like Oracle but faster' without AMP/PI/skew will not survive a DBA round.

90. What is an AMP?

1. Definition you should say first: Access Module Processor: a parallel worker that stores and processes part of the data. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.

3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.

91. What is a PE?

1. Definition you should say first: Parsing Engine: parses SQL, optimizes, and sends steps to AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PE = Parsing Engine: session handling, parse, optimize, dispatch steps to AMPs.

3. PE CPU spikes on bad dynamic SQL or too many short tactical requests. TASM protects mixed workloads.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT AccountName, UserName FROM DBC.SessionInfoV WHERE UserName = USER;
EXPLAIN SELECT COUNT(*) FROM edw.fact_orders;
```

5. People confuse PE with AMP. PE does not store the table; AMPs do.

92. What is BYNET?

1. Definition you should say first: The interconnect that moves messages and data between PEs and AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BYNET is the interconnect for messages, row redistribution, and duplication between PEs and AMPs.

3. Product joins and redistributing a huge uncompressed table flood BYNET. Check ResUsage and Viewpoint fabric metrics.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT *
FROM edw.fact_orders f
JOIN edw.dim_customer d ON d.region = f.region;
-- look for: redistributing / duplicating all rows of
```

5. If you only add CPU and ignore join geography, BYNET will still stall.

93. What is a vproc?

1. Definition you should say first: A virtual processor such as AMP or PE running on a node. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

94. What is a clique?

1. Definition you should say first: A set of nodes that share disks so a failed node can fail over. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

95. What is a hot standby node?

1. Definition you should say first: A spare node that can take over vprocs after a failure. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
```

```
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

96. What is Fallback?

1. Definition you should say first: A second copy of a table on another AMP for availability. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Fallback stores a second copy of rows on another AMP so a down AMP does not lose the table.

3. Enable on critical dimensions and control tables. It costs disk and write I/O. Not a substitute for BAR.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_calendar (
  cal_dt DATE NOT NULL
)
PRIMARY INDEX (cal_dt)
FALLBACK;
```

5. Fallback does not protect against a bad UPDATE. You still need backups and a back-out plan.

97. When do you use Fallback?

1. Definition you should say first: Critical tables that must survive AMP or disk loss. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Fallback stores a second copy of rows on another AMP so a down AMP does not lose the table.

3. Enable on critical dimensions and control tables. It costs disk and write I/O. Not a substitute for BAR.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_calendar (
  cal_dt DATE NOT NULL
)
PRIMARY INDEX (cal_dt)
FALLBACK;
```

5. Fallback does not protect against a bad UPDATE. You still need backups and a back-out plan.

98. What is a Primary Index?

1. Definition you should say first: The distribution key that decides which AMP stores a row. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.

3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

5. Changing PI means a full rebuild. Get it right on day one of the model.

99. Unique vs non-unique PI?

1. Definition you should say first: UPI enforces uniqueness; NUPI allows duplicates and can skew. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.

3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
```

```
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

5. Changing PI means a full rebuild. Get it right on day one of the model.

100. What is a Secondary Index?

1. Definition you should say first: An extra access path; it has a subtable and extra I/O. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.

3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.

101. What is a Join Index?

1. Definition you should say first: A stored join or aggregation that the optimizer can use. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.

3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.

102. What is PPI?

1. Definition you should say first: Partitioned Primary Index: partitions rows inside each AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.

3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.

103. What is a NoPI table?

1. Definition you should say first: A table without a PI; used for fast staging loads. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.

3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
```

```
) PRIMARY INDEX (order_id);
```

5. Changing PI means a full rebuild. Get it right on day one of the model.

104. What is a sparse map?

1. Definition you should say first: A map that uses a subset of AMPs for smaller tables. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.

3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.

4. Working code / syntax (write this on Teradata SQL):

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

5. Reconfig without a back-out window is a career incident.

105. What is a contiguous map?

1. Definition you should say first: The default map that uses all AMPs in the system. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.

3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.

4. Working code / syntax (write this on Teradata SQL):

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

5. Reconfig without a back-out window is a career incident.

106. What is hashing in Teradata?

1. Definition you should say first: The PI value is hashed to pick the AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.

3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

5. Changing PI means a full rebuild. Get it right on day one of the model.

107. What is skew?

1. Definition you should say first: Uneven row or CPU distribution across AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Skew is uneven rows or CPU across AMPs. Low-cardinality PI (country, flag, null-heavy key) causes it.

3. Compare AMP row counts, DBQL, Viewpoint skew. Fix PI, add a composite PI, or stage and redistribute on a better key.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW(pi_col))) AS amp_n, COUNT(*) AS rows
FROM edw.fact_orders
GROUP BY 1
ORDER BY 2 DESC;
-- skew ratio = max/avg
```

5. If NULLs hash together, a nullable PI becomes one giant AMP. Coalesce or pick another key.

108. How do you find skewed tables?

1. Definition you should say first: Compare AMP row counts or use DBQL and Viewpoint. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Skew is uneven rows or CPU across AMPs. Low-cardinality PI (country, flag, null-heavy key) causes it.

3. Compare AMP row counts, DBQL, Viewpoint skew. Fix PI, add a composite PI, or stage and redistribute on a better key.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW(pi_col))) AS amp_n, COUNT(*) AS rows
FROM edw.fact_orders
GROUP BY 1
ORDER BY 2 DESC;
-- skew ratio = max/avg
```

5. If NULLs hash together, a nullable PI becomes one giant AMP. Coalesce or pick another key.

109. What is a hot AMP?

1. Definition you should say first: An AMP doing more work than others, often from skew. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.

3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.

110. What is spool space?

1. Definition you should say first: Temporary space for intermediate query results. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

111. What is perm space?

1. Definition you should say first: Permanent table storage quota. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

112. What is temp space?

1. Definition you should say first: Space for global temporary tables. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
```

```
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

113. User vs database in Teradata?

1. Definition you should say first: A user can log on; a database is a space container. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

114. What is a profile in Teradata?

1. Definition you should say first: A set of account, spool, and password settings for users. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

115. What is a role?

1. Definition you should say first: A bundle of rights you grant to users. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

116. What is GRANT vs role?

1. Definition you should say first: GRANT is a privilege; a role groups many privileges. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

117. What is a macro?

1. Definition you should say first: A stored SQL snippet executed with EXEC. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

118. What is a stored procedure?

1. Definition you should say first: Procedural SQL with logic, not just a macro. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

119. What is a volatile table?

1. Definition you should say first: Session-scoped table that disappears on logoff. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

120. Global temporary vs volatile?

1. Definition you should say first: GTT definition is permanent; data is session-scoped. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

121. What is a derived table?

1. Definition you should say first: An inline subquery in the FROM clause. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

122. What is a CTE?

1. Definition you should say first: WITH clause query that names a result set. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

123. What is COLLECT STATISTICS?

1. Definition you should say first: Gather data demographics so the optimizer can plan well. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

124. When do you collect stats?

1. Definition you should say first: After large loads, skew, or bad plans; not blindly every hour. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

125. What is SUMMARY stats?

1. Definition you should say first: A cheaper stats option on some releases for quick demographics. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

126. What is sampled stats?

1. Definition you should say first: Stats on a sample; faster but less accurate. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

127. How do you see a plan?

1. Definition you should say first: EXPLAIN SELECT ... Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

128. What is a confidence level in EXPLAIN?

1. Definition you should say first: How sure the optimizer is about row estimates. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

129. What is a product join?

1. Definition you should say first: Every row compared to every row; often a warning sign. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

130. What is a merge join?

1. Definition you should say first: Join on sorted/hashed keys; usually cheaper. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

131. What is a hash join?

1. Definition you should say first: Build a hash table of the smaller set and probe it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

132. What is a nested join?

1. Definition you should say first: Index-driven lookup join. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

133. What is redistribution?

1. Definition you should say first: Moving rows across AMPs so join keys co-locate. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
```

```
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

134. What is duplication of a table in a join?

1. Definition you should say first: Copying a small table to all AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

135. What is a residual condition?

1. Definition you should say first: A filter applied after the join or retrieve step. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

136. What is a lock in Teradata?

1. Definition you should say first: Access, read, write, or exclusive lock on a table or row hash. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

137. What is a deadlock?

1. Definition you should say first: Two sessions waiting on each other; one is aborted. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

138. How do you reduce lock contention?

1. Definition you should say first: Shorter transactions, nightly batch windows, and row-hash locks. Then spend the rest of

the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

139. What is a transaction in Teradata?

1. Definition you should say first: BT/ET or ANSI transaction around SQL. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BTET (Teradata mode) auto-commits each request unless BT/ET. ANSI uses BEGIN/COMMIT. A request is one SQL; a transaction can be many.

3. ETL scripts should be explicit: BT; steps; ET; with error handling in BTEQ.

4. Working code / syntax (write this on Teradata SQL):

```
.SET SESSION TRANSACTION BTET
BT;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-01';
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
ET;
```

5. A failed INSERT after DELETE without a transaction leaves an empty partition. That is a Sev1.

140. BTET vs ANSI mode?

1. Definition you should say first: BTET is Teradata transaction mode; ANSI is commit-oriented. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BTET (Teradata mode) auto-commits each request unless BT/ET. ANSI uses BEGIN/COMMIT. A request is one SQL; a transaction can be many.

3. ETL scripts should be explicit: BT; steps; ET; with error handling in BTEQ.

4. Working code / syntax (write this on Teradata SQL):

```
.SET SESSION TRANSACTION BTET
BT;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-01';
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
ET;
```

5. A failed INSERT after DELETE without a transaction leaves an empty partition. That is a Sev1.

141. What is FastLoad?

1. Definition you should say first: Utility to load empty tables quickly with two phases. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

142. FastLoad limitation?

1. Definition you should say first: Target must be empty; no SI during load in classic use. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

143. What is MultiLoad?

1. Definition you should say first: Utility for insert/update/delete on populated tables. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

144. What is TPump?

1. Definition you should say first: Low-volume continuous load with row-hash locks. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

145. What is FastExport?

1. Definition you should say first: High-volume export from Teradata. Then spend the rest of the answer on architecture, a

working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

146. What is BTEQ?

1. Definition you should say first: CLI for SQL, scripts, and simple imports. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

147. What is TPT?

1. Definition you should say first: Teradata Parallel Transporter: modern load/export framework. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

148. TPT vs legacy utilities?

1. Definition you should say first: TPT can replace FastLoad/MultiLoad/FastExport with operators. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

149. What is a load slot?

1. Definition you should say first: A limit on concurrent load jobs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

150. What is a checkpoint in MultiLoad?

1. Definition you should say first: A restart point after a failure. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

151. What is an error table?

1. Definition you should say first: Where utilities land rejected rows. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV

tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

152. What is a UV table?

1. Definition you should say first: Uniqueness violation table in MultiLoad. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

153. What is Viewpoint?

1. Definition you should say first: Web console for health, queries, and alerts. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

154. What is QueryGrid?

1. Definition you should say first: Fabric to query Hadoop, Oracle, and other systems from Teradata. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

155. What is TASM?

1. Definition you should say first: Teradata Active System Management: workload rules and throttles. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

156. What is a workload in TASM?

1. Definition you should say first: A class of queries with priority and limits. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

157. What is a throttle?

1. Definition you should say first: A cap on concurrency for a workload. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

158. What is a filter in TASM?

1. Definition you should say first: A rule that rejects or delays some queries. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

159. What is exception handling in TASM?

1. Definition you should say first: Move or abort queries that exceed CPU or skew. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

160. What is WD-Default?

1. Definition you should say first: The default workload when nothing else matches. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

161. What is a utility session?

1. Definition you should say first: Sessions used by load/export utilities. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

162. What is BAR?

1. Definition you should say first: Backup and restore, often with DSA or NetBackup. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

163. What is DSA?

1. Definition you should say first: Data Stream Architecture for backups. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

164. Full vs incremental backup?

1. Definition you should say first: Full copies everything; incremental copies changes. Then spend the rest of the answer on

architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

165. How do you restore one table?

1. Definition you should say first: Use BAR object-level restore if the backup supports it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

166. What is a dump vs backup?

1. Definition you should say first: Dump often means a system dump for support; backup is data protection. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

167. What is PDBA vs production DBA?

1. Definition you should say first: PDBA may own platform; production DBA owns day-to-day jobs and users. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

168. What is space reclamation?

1. Definition you should say first: Freeing perm after deletes; packing and fallback copies matter. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
```

```
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

169. What is a cylinder?

1. Definition you should say first: A unit of disk allocation on an AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

170. What is a table header?

1. Definition you should say first: Metadata stored with the table on each AMP. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

171. What is a journal?

1. Definition you should say first: Before/after image used for recovery on some tables. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

172. Permanent journal vs transient?

1. Definition you should say first: Permanent is extra protection; transient is for transactions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

173. What is TVS?

1. Definition you should say first: Teradata Virtual Storage: storage virtualization layer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
```

```
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

174. What is FSYS?

1. Definition you should say first: File system layer under AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

175. What is a node crash recovery?

1. Definition you should say first: Vprocs migrate; cliques and HSN determine downtime. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

176. What is a database restart?

1. Definition you should say first: TPA reset: sessions drop; need to reconnect. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

177. What is TPA?

1. Definition you should say first: Teradata parallel database process stack. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

178. What is PDE?

1. Definition you should say first: Parallel Database Extensions: OS layer for vprocs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
```

```
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

179. What is a gateway?

1. Definition you should say first: Connection manager for client sessions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

180. What is a session?

1. Definition you should say first: A logged-on PE connection from a client. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

181. Default session mode?

1. Definition you should say first: Depends on client; know if you are ANSI or Teradata mode. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

182. What is a request vs a transaction?

1. Definition you should say first: A request is one SQL; a transaction may be many requests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BTET (Teradata mode) auto-commits each request unless BT/ET. ANSI uses BEGIN/COMMIT. A request is one SQL; a transaction can be many.

3. ETL scripts should be explicit: BT; steps; ET; with error handling in BTEQ.

4. Working code / syntax (write this on Teradata SQL):

```
.SET SESSION TRANSACTION BTET
BT;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-01';
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
ET;
```

5. A failed INSERT after DELETE without a transaction leaves an empty partition. That is a Sev1.

183. How do you kill a bad query?

1. Definition you should say first: Abort in Viewpoint or with a console abort after checking impact. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

184. How do you find who filled the spool?

1. Definition you should say first: DBQL plus user spool limits. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

185. What is DBQL?

1. Definition you should say first: Database Query Log: history of queries and steps. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

186. What is ResUsage?

1. Definition you should say first: Resource usage macros for CPU, disk, and BYNET. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

187. What is a heart beat in Viewpoint?

1. Definition you should say first: System health pulse; red means investigate now. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

188. What is a blocked session?

1. Definition you should say first: Waiting on a lock held by another session. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row

lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

189. How do you see blockers?

1. Definition you should say first: Lock viewer in Viewpoint or lock query views. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

190. What is a deadlock timeout?

1. Definition you should say first: How long Teradata waits before aborting a deadlock victim. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

191. What is a PPI advantage?

1. Definition you should say first: Partition elimination so fewer cylinders are read. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.

3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.

192. What is a character PPI?

1. Definition you should say first: Partitioning on a character column; less common than date PPI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.

3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in

WHERE.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.

193. What is multilevel PPI?

1. Definition you should say first: More than one partition expression. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.

3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.

194. What is a column partition (CP)?

1. Definition you should say first: Columnar storage for analytic tables on some releases. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.

3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.

195. What is Hybrid Row/Column?

1. Definition you should say first: Mix of row and column partitions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PPI partitions rows inside each AMP (often by date) so scans prune. Character PPI is possible but date/int is cleaner. CP / hybrid are columnar options for analytic compression.

3. Partition facts by order_dt RANGE_N. Collect stats on the partition column. Avoid functions on the partition column in WHERE.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE NOT NULL,
  amt DECIMAL(18,2)
)
PRIMARY INDEX (order_id)
PARTITION BY RANGE_N(order_dt BETWEEN DATE '2018-01-01' AND DATE '2028-12-31' EACH INTERVAL '1' MONTH);
```

5. WHERE EXTRACT(YEAR FROM order_dt)=2026 kills pruning. Use a date range.

196. What is compression in Teradata?

1. Definition you should say first: MVC, UDT, algorithmic, or block-level depending on release. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.

3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

5. Compressing a near-unique column wastes CPU for no disk win.

197. What is MVC?

1. Definition you should say first: Multi-Value Compression: dictionary compression on a column. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.

3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

5. Compressing a near-unique column wastes CPU for no disk win.

198. What is a UDT?

1. Definition you should say first: User-defined type. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.

3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

5. Compressing a near-unique column wastes CPU for no disk win.

199. What is a UDF?

1. Definition you should say first: User-defined function. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.

3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
```

```
) AS nos_d;
```

5. Compressing a near-unique column wastes CPU for no disk win.

200. What is a table operator?

1. Definition you should say first: SQL-HMR style operator for custom processing. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.

3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

5. Compressing a near-unique column wastes CPU for no disk win.

201. What is Native Object Store (NOS)?

1. Definition you should say first: Read/write object storage (S3-style) from Teradata Vantage. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. MVC = multi-value compression on repeating values. UDT/UDF extend types/functions. Table operators and NOS read object store (S3-style) from SQL.

3. Compress low-cardinality columns. NOS for lake files; Teradata remains the governed EDW.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE TABLE edw.dim_status (
  status_cd CHAR(3) COMPRESS ('NEW','OPN','CLS'),
  status_desc VARCHAR(40)
) PRIMARY INDEX (status_cd);
SELECT $PATH, col1 FROM (
  LOCATION '/s3/bucket/path'
) AS nos_d;
```

5. Compressing a near-unique column wastes CPU for no disk win.

202. What is Vantage?

1. Definition you should say first: Teradata's analytics platform branding for newer releases. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata is a shared-nothing MPP analytic database. Vantage is the current platform branding (SQL plus analytics). Each AMP owns a slice of every table.

3. Draw PE -> BYNET -> AMPs. Mention PI, stats, TASM, and BAR in the first minute.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.DiskSpaceV WHERE DatabaseName = 'EDW';
HELP SESSION;
SELECT HASHAMP() AS amps;
```

5. Calling it 'just like Oracle but faster' without AMP/PI/skew will not survive a DBA round.

203. What is QueryBand?

1. Definition you should say first: Name/value tags on a session or transaction for workload and audit. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

204. What is an account string?

1. Definition you should say first: Account used for chargeback and TASM mapping. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

205. What is a performance group?

1. Definition you should say first: Older priority scheme; TASM workloads replaced much of this. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

206. What is a resource partition?

1. Definition you should say first: A slice of CPU reserved for a set of workloads. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

207. What is tactical vs decision support?

1. Definition you should say first: Tactical is short; DSS is heavy reporting. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

208. How do you protect tactical queries?

1. Definition you should say first: TASM: high priority, low throttle, and exception on long CPU. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
```

```
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

209. What is a delayed query?

1. Definition you should say first: Queued by a throttle until a slot is free. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

210. What is a reject in TASM?

1. Definition you should say first: The query is not allowed to run. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

211. What is an object lock vs row hash lock?

1. Definition you should say first: Object is table-level; row hash is finer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

212. What is a dummy row lock?

1. Definition you should say first: Locking a hash with no row; still blocks that hash. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

213. How do you load a large fact table daily?

1. Definition you should say first: Stage NoPI or FastLoad, then INSERT-SELECT with collect stats. Then spend the rest of

the answer on architecture, a working example, and a production failure.

2. Teradata is shared-nothing MPP: PE parses, BYNET ships, AMPs own their rows and spool.

3. Always name PI, stats, skew, spool, and the utility you would use. Then EXPLAIN before you run in prod.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT * FROM edw.fact_orders
WHERE order_dt = DATE '2026-09-01';
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id, order_dt);
```

5. A product join plus missing stats plus a bad NUPI will light up one hot AMP and fill spool.

214. What is an INSERT-SELECT?

1. Definition you should say first: Load from one table into another set-based. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

215. What is MERGE INTO?

1. Definition you should say first: Upsert based on a match condition. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

216. What is UPDATE from a join?

1. Definition you should say first: Set-based update using another table. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
```

```
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

217. What is a SET vs MULTISET table?

1. Definition you should say first: SET rejects duplicate rows; MULTISET allows them. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SET tables reject duplicate rows (expensive duplicate row check). MULTISET allows duplicates and is the default for most ETL facts.

3. Use SET only when uniqueness must be enforced by the table and volume is modest. Prefer UPI or USI + MULTISET for facts.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE SET TABLE edw.dim_code (
  code CHAR(4) NOT NULL,
  descr VARCHAR(40)
) UNIQUE PRIMARY INDEX (code);
CREATE MULTISET TABLE edw.fact_click (click_id BIGINT, url VARCHAR(200)) PRIMARY INDEX (click_id);
```

5. A SET fact with no UPI will spend CPU comparing rows on every insert.

218. Duplicate row check cost?

1. Definition you should say first: SET tables can be expensive on NUPI during inserts. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SET tables reject duplicate rows (expensive duplicate row check). MULTISET allows duplicates and is the default for most ETL facts.

3. Use SET only when uniqueness must be enforced by the table and volume is modest. Prefer UPI or USI + MULTISET for facts.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE SET TABLE edw.dim_code (
  code CHAR(4) NOT NULL,
  descr VARCHAR(40)
) UNIQUE PRIMARY INDEX (code);
CREATE MULTISET TABLE edw.fact_click (click_id BIGINT, url VARCHAR(200)) PRIMARY INDEX (click_id);
```

5. A SET fact with no UPI will spend CPU comparing rows on every insert.

219. What is a queue table?

1. Definition you should say first: A table with consume semantics for messaging-style pulls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

220. What is an identity column?

1. Definition you should say first: System-generated numbers; be careful with distribution. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Macro = parameterized SQL. SP = procedural. Volatile = session table. GTT = permanent def, session data. Derived = FROM (SELECT...). CTE = WITH. Queue tables support FIFO. Identity is a generated number, not a PI by default thinking.

3. Use volatile for intermediate in ETL scripts. Do not hide 2000-line business logic only in SPs if the team cannot EXPLAIN it.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VOLATILE TABLE vt_today AS (
  SELECT * FROM edw.fact_orders WHERE order_dt = CURRENT_DATE
) WITH DATA
PRIMARY INDEX (order_id)
ON COMMIT PRESERVE ROWS;
WITH c AS (SELECT cust_id FROM edw.dim_customer WHERE segment = 'GOLD')
SELECT * FROM edw.fact_orders f JOIN c ON c.cust_id = f.cust_id;
```

5. Volatile tables disappear on logoff - BTEQ scripts that reconnect lose them.

221. What is a date PI problem?

1. Definition you should say first: Low cardinality dates skew if used as PI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.

3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

5. Changing PI means a full rebuild. Get it right on day one of the model.

222. Good PI for a fact table?

1. Definition you should say first: A high-cardinality join key, often a customer or transaction id. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PI is the distribution key: HASHROW -> AMP. UPI unique, NUPI allows dupes. NoPI is unhashed staging. Date-only PI often skews. Fact PI is usually a high-cardinality id (order_id), not a status flag.

3. Choose PI for join geography and even AMP counts, not for 'it looks like a PK'. Stage on NoPI, then INSERT-SELECT to the PI table.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE MULTISET TABLE stg.orders_in (
  order_id BIGINT, order_dt DATE, amt DECIMAL(18,2)
) NO PRIMARY INDEX;
CREATE MULTISET TABLE edw.fact_orders (
  order_id BIGINT NOT NULL,
  order_dt DATE,
  amt DECIMAL(18,2)
) PRIMARY INDEX (order_id);
```

5. Changing PI means a full rebuild. Get it right on day one of the model.

223. What is referential integrity in Teradata?

1. Definition you should say first: Optional; soft RI is often used for optimizer hints. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
```

```
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

224. What is a batch vs tactical window?

1. Definition you should say first: Batch at night with loads; tactical during business hours. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata is shared-nothing MPP: PE parses, BYNET ships, AMPs own their rows and spool.

3. Always name PI, stats, skew, spool, and the utility you would use. Then EXPLAIN before you run in prod.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT * FROM edw.fact_orders
WHERE order_dt = DATE '2026-09-01';
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id, order_dt);
```

5. A product join plus missing stats plus a bad NUPI will light up one hot AMP and fill spool.

225. How do you handle a failed FastLoad?

1. Definition you should say first: Drop or empty the target, fix the file, restart; check error tables. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

226. What is a two-phase FastLoad?

1. Definition you should say first: Phase 1 loading to AMPs; phase 2 sorting into the table. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

227. Can FastLoad load a PI with SI?

1. Definition you should say first: Classic FastLoad wants no secondary indexes on the target. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

228. What is a MiniBatch?

1. Definition you should say first: A pattern of small MultiLoad or TPT jobs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

229. What is streaming ingest?

1. Definition you should say first: TPump or TPT Stream for near-real-time. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. FastLoad = empty table, two phases, no SI, max one PI. MultiLoad = maintained tables, upsert, checkpoints, error/UV tables. TPump = low volume trickle. FastExport = out. BTEQ = script/SQL. TPT = modern operator framework. INSERT-SELECT and MERGE are SQL loads.

3. Nightly facts: TPT Load or FastLoad to NoPI/stg then INSERT-SELECT. Failed FastLoad: drop/release lock, clean err tables, restart from checkpoint rules.

4. Working code / syntax (write this on Teradata SQL):

```
/* TPT Load idea */
-- DEFINE OPERATOR DDL / LOAD
-- FastLoad limits: empty table, no SI, no PPI in old rules (version-specific)
INSERT INTO edw.fact_orders
SELECT * FROM stg.orders_in;
MERGE INTO edw.dim_customer t
USING stg.customer s ON t.cust_id = s.cust_id
WHEN MATCHED THEN UPDATE SET name = s.name
WHEN NOT MATCHED THEN INSERT (cust_id, name) VALUES (s.cust_id, s.name);
```

5. Leaving a FastLoad lock on a table blocks the morning. Always script the release/cleanup path.

230. How do you grant access to a new analyst?

1. Definition you should say first: Role-based: create user, grant role, set spool and profile. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

231. How do you revoke a leaver?

1. Definition you should say first: Lock the user, revoke roles, drop or keep for audit per policy. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

232. What is a password lockout?

1. Definition you should say first: Profile setting after failed logons. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

233. What is LDAP vs TD database auth?

1. Definition you should say first: Enterprise directory vs Teradata passwords. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

234. What is SSO to Viewpoint?

1. Definition you should say first: Viewpoint can use corporate identity; still map TD users. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A profile holds password/spool defaults. A role is a bundle of rights. GRANT to roles, not 200 users. LDAP/SSO for people; TD passwords for service ids.

3. Joiner: create user, grant role_analyst, set profile. Leaver: revoke role, lock, then drop after HR date. Never share DBC.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE USER u_jaya FROM edw_users AS PASSWORD = *** PERM = 0 SPOOL = 5e9 PROFILE = pr_analyst;
GRANT role_read_edw TO u_jaya;
REVOKE role_read_edw FROM u_ex;
MODIFY USER u_ex AS PASSWORD = *** LOCKED = U;
```

5. Personal copies of prod tables in a sandbox with PII is an audit finding.

235. What is a zone in Viewpoint?

1. Definition you should say first: A filtered view of systems for a team. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

236. What is a portlet?

1. Definition you should say first: A Viewpoint dashboard widget. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Viewpoint is the ops UI (portlets, zones, heartbeat). TASM is workload management: workloads, throttles, filters, exceptions. Tactical = short SLA; DSS = heavy. QueryBand tags sessions. QueryGrid talks to other platforms.

3. Put tactical in a high-priority partition with a throttle on DSS. Reject or delay runaways. Set QueryBand in ETL for chargeback.

4. Working code / syntax (write this on Teradata SQL):

```
SET QUERY_BAND = 'App=ETL;Job=FACT_ORDERS;Team=EDW;' FOR SESSION;
-- TASM: WD_Tactical timeout 10s, throttle DSS to N queries
-- Viewpoint alert: heartbeat down, blocked sessions > 20, spool > 80%
```

5. Everything in WD-Default means TASM is not actually implemented.

237. What is a system heartbeat alert?

1. Definition you should say first: Pager-worthy if the database is down. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata is shared-nothing MPP: PE parses, BYNET ships, AMPs own their rows and spool.

3. Always name PI, stats, skew, spool, and the utility you would use. Then EXPLAIN before you run in prod.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT * FROM edw.fact_orders
WHERE order_dt = DATE '2026-09-01';
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id, order_dt);
```

5. A product join plus missing stats plus a bad NUPI will light up one hot AMP and fill spool.

238. How do you plan an upgrade?

1. Definition you should say first: Read the release, test BAR, check views, schedule an outage window. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Upgrades need compat checks, BAR, a freeze, and a back-out. Stall in Viewpoint is wait time, not always a down system.

3. Read release notes for TPT/TASM changes. Run CHECKTABLE only per vendor guidance.

4. Working code / syntax (write this on Teradata SQL):

```
-- pre-upgrade
-- 1 BAR full  2 capture TASM  3 freeze  4 upgrade  5 validate Viewpoint heartbeat  6 smoke FastLoad + SELECT HASHAMP()
```

5. Skipping a backup because 'it is only a patch' is the story in half of DBA incident talks.

239. What is a compat view?

1. Definition you should say first: Older dictionary views kept for scripts. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.

3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

5. Updating DBC base tables by hand is how you corrupt a system.

240. DBC vs SYSLIB?

1. Definition you should say first: DBC is the data dictionary; SYSLIB holds functions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.

3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

5. Updating DBC base tables by hand is how you corrupt a system.

241. What is DBC.TablesV?

1. Definition you should say first: Dictionary view of tables. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.

3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

5. Updating DBC base tables by hand is how you corrupt a system.

242. How do you find table size?

1. Definition you should say first: SUM of currentperm from DBC.TableSizeV grouped by table. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.

3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

5. Updating DBC base tables by hand is how you corrupt a system.

243. How do you find user space?

1. Definition you should say first: DBC.DiskSpaceV or Viewpoint space portlet. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DBC is the dictionary. SYSLIB has functions. *V views are the supported interface. Always query views, not legacy base tables, unless PDBA asks.

3. Size from TableSizeV / DiskSpaceV. Space from DiskSpaceV CurrentPerm vs MaxPerm.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT DatabaseName, TableName, SUM(CurrentPerm) AS bytes
FROM DBC.TableSizeV
WHERE DatabaseName = 'EDW'
GROUP BY 1, 2
ORDER BY 3 DESC;
SELECT TableName, TableKind FROM DBC.TablesV WHERE DatabaseName = 'EDW';
```

5. Updating DBC base tables by hand is how you corrupt a system.

244. What is currentperm vs peakperm?

1. Definition you should say first: Current used vs high-water used. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Perm = table bytes. Spool = query scratch. Temp = GTT data. Users log on; databases hold objects. PeakPerm is high-water. Reclamation frees deleted space over time.

3. When spool exceeds, find the query in DBQL, EXPLAIN, add stats, or raise spool with a ticket - do not silently grant unlimited.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT UserName, PeakSpool FROM DBC.DiskSpaceV WHERE PeakSpool > 0 ORDER BY 2 DESC;
SELECT DatabaseName, SUM(CurrentPerm) FROM DBC.TableSizeV GROUP BY 1;
-- abort: ABORT SESSION <host>, <session>
```

5. Granting huge spool to hide a product join is not tuning.

245. What is a stall?

1. Definition you should say first: AMPs waiting on I/O or each other; check ResUsage. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

246. What is a bottleneck on BYNET?

1. Definition you should say first: Heavy redistribution; redesign PI or join. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BYNET is the interconnect for messages, row redistribution, and duplication between PEs and AMPs.

3. Product joins and redistributing a huge uncompressed table flood BYNET. Check ResUsage and Viewpoint fabric metrics.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT *
FROM edw.fact_orders f
JOIN edw.dim_customer d ON d.region = f.region;
-- look for: redistributing / duplicating all rows of
```

5. If you only add CPU and ignore join geography, BYNET will still stall.

247. What is a CPU-bound query?

1. Definition you should say first: High AMP CPU; check product joins and functions on columns. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Sessions enter via gateway. Default mode is often BTET (site-specific). DBQL logs queries. ResUsage is resource telemetry. Stall = waiting (locks, I/O). CPU-bound = AMP CPU high with little I/O wait.

3. Kill with abort session after you capture EXPLAIN/DBQL. Do not kill a BAR or a FastLoad without the utility owner.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT SessionNo, UserName, StatementText FROM DBC.QryLogV
WHERE StartTime > CURRENT_TIMESTAMP - INTERVAL '10' MINUTE;
ABORT SESSION 1, 1042;
```

5. Killing the wrong session during MERGE can leave a transaction open. Check SessionInfo first.

248. What is a missing stats symptom?

1. Definition you should say first: Wrong join type and huge spool. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

249. How often to collect stats on a large fact?

1. Definition you should say first: After major loads; use THRESHOLD where available. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

250. What is stats THRESHOLD?

1. Definition you should say first: Skip collect if data has not changed enough. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Stats are demographics for the optimizer. Collect after large loads or skew. Sampled/summary are cheaper. THRESHOLD skips collect if little change. Huge facts: collect on PI, partition, join columns - not every column daily.

3. Help stats, compare EXPLAIN estimates vs actual DBQL. Recollect when a job's plan flips.

4. Working code / syntax (write this on Teradata SQL):

```
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_id);
COLLECT STATISTICS ON edw.fact_orders COLUMN (order_dt);
COLLECT STATISTICS USING SAMPLE ON edw.stg_big COLUMN (load_dt);
HELP STATISTICS ON edw.fact_orders;
```

5. Collecting all columns hourly on a 20 TB fact is a worse outage than stale stats.

251. What is a copy job in BAR?

1. Definition you should say first: Copy backup data to another system. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

252. What is a restore to a different system?

1. Definition you should say first: Needs compatible maps and enough AMPs or a remap strategy. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. BAR = backup and restore. DSA is the modern BAR. Full vs incremental. Dump is often a looser term; production uses catalogued BAR jobs. Restore one table for a bad load; restore to another system for DR tests.

3. Nightly incremental + weekly full. Test a table restore quarterly. Document the job name in the runbook.

4. Working code / syntax (write this on Teradata SQL):

```
-- DSA / BAR job (logical)
JOB: BAR_EDW_DAILY
  INCLUDE DATABASE edw
  INCREMENTAL
RESTORE TABLE edw.fact_orders FROM BAR_EDW_DAILY WHERE BACKUP_DATE = DATE '2026-09-02';
```

5. A backup that never had a restore drill is not a backup. Interviewers ask when you last restored.

253. What is a map change?

1. Definition you should say first: Redistribute tables after adding AMPs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.

3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.

4. Working code / syntax (write this on Teradata SQL):

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

5. Reconfig without a back-out window is a career incident.

254. What is a reconfig?

1. Definition you should say first: System change such as adding nodes; planned outage. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Contiguous map = all AMPs. Sparse map = subset for small tables. Map change / reconfig redistributes objects after adding AMPs.

3. Do not put a 2-row calendar on 1000 AMPs without a sparse map if your version supports it. Plan reconfig with BAR and a freeze.

4. Working code / syntax (write this on Teradata SQL):

```
SHOW TABLE edw.dim_calendar;
-- check MAP in newer Vantage
SELECT DatabaseName, TableName, MapName FROM DBC.TablesV WHERE TableName = 'fact_orders';
```

5. Reconfig without a back-out window is a career incident.

255. What is HSN vs clique failover?

1. Definition you should say first: HSN is a spare node; clique shares disks among nodes. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

256. What is a down AMP?

1. Definition you should say first: AMP not processing; fallback or HSN may cover. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.

3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.

257. What is a catchup?

1. Definition you should say first: Recovering fallback copies after a component returns. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. vproc = virtual processor (AMP/PE). Clique = nodes sharing disks. HSN = spare node. PDE/TPA = OS/DB layers. Gateway accepts sessions. Catchup replays after AMP recovery.

3. On node loss, vprocs migrate in the clique or onto HSN. Users see a restart window. Check Viewpoint heartbeat.

4. Working code / syntax (write this on Teradata SQL):

```
-- After restart:
SELECT * FROM DBC.VprocStatus;
SELECT * FROM DBC.DatabaseStatus;
```

5. No fallback + down AMP + no clique spare = table inaccessible. Know your availability design.

258. What is a checktable?

1. Definition you should say first: Utility to check table integrity. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

259. What is a scandisk?

1. Definition you should say first: Lower-level disk integrity check. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

260. What is a perm cache?

1. Definition you should say first: Memory cache of perm data; watch hit rates. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PE = Parsing Engine: session handling, parse, optimize, dispatch steps to AMPs.

3. PE CPU spikes on bad dynamic SQL or too many short tactical requests. TASM protects mixed workloads.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT AccountName, UserName FROM DBC.SessionInfoV WHERE UserName = USER;
EXPLAIN SELECT COUNT(*) FROM edw.fact_orders;
```

5. People confuse PE with AMP. PE does not store the table; AMPs do.

261. What is FSG cache?

1. Definition you should say first: File segment cache for I/O. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

262. What is a TVS temperature?

1. Definition you should say first: Hot/cold data placement on mixed storage. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PDBA is platform DBA (OS, PDE, BAR). Cylinders/FSG/TVS are storage layers and temperature-aware placement. Journals: transient for tx, permanent optional. Checktable/scandisk are integrity tools used with care.

3. You will not run scandisk on a hunch in prod. You will watch TVS temperature and FSG hit rate with the PDBA.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT * FROM DBC.StorageSizeV SAMPLE 10;
-- FSG cache hit: ResUsage / Viewpoint memory portlet
-- Permanent journal: show table will list JOURNAL options
```

5. Running checktable on a live 100 TB fact without a window can be an outage.

263. What is a 1-AMP operation?

1. Definition you should say first: A query that hits a single AMP via unique PI access. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.

3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.

264. What is an all-AMP retrieve?

1. Definition you should say first: Every AMP reads its piece of the table. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.

3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.

265. What is a group AMP operation?

1. Definition you should say first: A subset of AMPs, for example sparse maps. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. AMP = Access Module Processor. It stores rows, executes steps, and uses spool. A hot AMP does extra work (skew). Down AMP triggers fallback/clique recovery. 1-AMP ops hit one AMP; all-AMP retrieve hits every AMP.

3. Check AMP CPU and row counts when a query is slow. Hash the PI to see placement.

4. Working code / syntax (write this on Teradata SQL):

```
SELECT HASHAMP(HASHBUCKET(HASHROW('1001'))) AS amp_guess;
SELECT vproc, COUNT(*) FROM edw.fact_orders GROUP BY 1 ORDER BY 2 DESC;
-- Viewpoint: AMP usage / skew
```

5. A UPI on gender or date can put millions of rows on one AMP. That is the classic hot AMP.

266. What is a join index maintenance cost?

1. Definition you should say first: Extra work on every base-table change. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.

3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.

267. When is a JI worth it?

1. Definition you should say first: Stable, repeated joins that dominate CPU. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.

3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.

268. What is an aggregate JI?

1. Definition you should say first: Pre-aggregated stored result. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.

3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.

269. What is a single-table JI?

1. Definition you should say first: A stored projection or extra access path on one table. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A Join Index is a stored join or aggregation the optimizer can pick. It costs insert/update/delete maintenance.

3. Worth it when a dashboard repeats the same join and you cannot change the app SQL. Recollect stats on the JI too.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE JOIN INDEX edw.ji_order_cust AS
SELECT f.order_id, f.amt, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
PRIMARY INDEX (order_id);
```

5. A JI that is never chosen in EXPLAIN is dead weight. Drop it.

270. What is a covering index?

1. Definition you should say first: An index that satisfies a query without the base table. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.

3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.

271. What is a value-ordered NUSI?

1. Definition you should say first: NUSI stored in value order for range access. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.

3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.

272. What is a hash-ordered NUSI?

1. Definition you should say first: Default NUSI organization. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.

3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.

273. What is a unique secondary index?

1. Definition you should say first: Enforces uniqueness and supports lookups. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.

3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.

274. Cost of USI on load?

1. Definition you should say first: Load utilities may require dropping USI first. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. SI is an extra access path with a subtable. USI enforces uniqueness. NUSI can be value-ordered or hash-ordered. Covering means the SI answers the query without the base row. USI/NUSI slow loads.

3. Add SI only for a proven tactical access path. Drop unused SI after DBQL review.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE UNIQUE INDEX usi_cust_email (email) ON edw.dim_customer;
CREATE INDEX nusi_order_dt (order_dt) ON edw.fact_orders;
```

5. A USI on a FastLoad target is illegal or forces a slower path. Know utility limits.

275. What is a soft RI?

1. Definition you should say first: Optimizer uses the relationship without enforcing it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

276. What is a batch error table?

1. Definition you should say first: Landing for failed rows in a batch. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

277. How do you handle late-arriving dimensions?

1. Definition you should say first: Staging and MERGE; do not invent keys. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
```

```
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

278. What is slowly changing dimension?

1. Definition you should say first: Type 1 overwrite vs Type 2 history rows. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

279. How does a DBA support SCD2?

1. Definition you should say first: PI, PPI on dates, and space for history. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

280. What is a surrogate key?

1. Definition you should say first: Internal integer key; pick a PI that does not skew. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

281. What is a natural key?

1. Definition you should say first: Business key such as account number. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
```

```
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

282. What is 3NF vs star schema?

1. Definition you should say first: 3NF is normalized EDW; star is dimensional mart. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

283. Where does Teradata sit in a lakehouse?

1. Definition you should say first: Often the governed EDW; NOS can query the lake. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

284. What is data lineage?

1. Definition you should say first: Where a column came from; DBA supports via views and docs. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

285. What is a view vs a table?

1. Definition you should say first: View is stored SQL; table is stored rows. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then

an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

286. What is a recursive view?

1. Definition you should say first: View that refers to itself; use carefully. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Teradata RI can be hard or soft. SCD2 keeps history with surrogate keys. Star vs 3NF is mart vs EDW. NOS/lake sits beside the EDW. Views store SQL, not rows.

3. DBA supports SCD2 with space, PI on surrogate, PPI on dates, and stats. Late dimensions get a default unknown key then an update.

4. Working code / syntax (write this on Teradata SQL):

```
CREATE VIEW edw.v_orders AS
SELECT f.*, c.segment
FROM edw.fact_orders f
JOIN edw.dim_customer c ON c.cust_id = f.cust_id;
-- SCD2 row:
-- cust_sk, cust_id, name, start_dt, end_dt, is_current
```

5. Recursive views without a depth cap can spool out. Hard RI on a 2-billion row fact can destroy load windows.

287. What is a locking view?

1. Definition you should say first: A view that includes LOCKING modifiers. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

288. LOCKING ROW FOR ACCESS?

1. Definition you should say first: Allows dirty reads to reduce blocking. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

289. When is ACCESS locking dangerous?

1. Definition you should say first: When the reader must see committed consistent numbers. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

290. What is a deadlock victim?

1. Definition you should say first: The session Teradata aborts to break a deadlock. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Locks: row hash, table, database. ACCESS locking reads dirty. Deadlock: two sessions wait; one is victim. Dummy row lock shows up on some dictionary ops.

3. Use LOCKING ROW FOR ACCESS on reports that can stand dirty reads. For finance numbers, READ and a proper window. Viewpoint blocked sessions shows the blocker.

4. Working code / syntax (write this on Teradata SQL):

```
LOCKING ROW FOR ACCESS
SELECT SUM(amt) FROM edw.fact_orders WHERE order_dt = CURRENT_DATE;
-- find blockers via Viewpoint or DBC.LockInfoV / SessionInfo
ABORT SESSION hostid, sessionid;
```

5. ACCESS during a load can show half-committed totals to a CFO dashboard. Know when that is unacceptable.

291. How do you tune a product join?

1. Definition you should say first: Stats, better join keys, or rewrite to merge/hash. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. EXPLAIN is the plan. Confidence is estimate quality. Merge/hash are good when keys match. Product join is cartesian-ish. Redistribute vs duplicate is how join geography is built. Residual is leftover filter.

3. Fix product joins with stats, better ON keys, or rewrite. Look for 'duplicating all rows of a 2-billion row table'.

4. Working code / syntax (write this on Teradata SQL):

```
EXPLAIN
SELECT f.order_id, c.segment
FROM edw.fact_orders f
INNER JOIN edw.dim_customer c ON c.cust_id = f.cust_id
WHERE f.order_dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31';
```

5. SET tables with duplicate checks plus a product join will run until spool dies.

292. What is a character set issue?

1. Definition you should say first: Latin vs Unicode; conversions can cost CPU. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

293. What is a case-specific compare?

1. Definition you should say first: CS vs CASESPECIFIC in comparisons. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

294. What is a format in BTEQ?

1. Definition you should say first: How numbers and dates print in reports. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

295. What is .IF ERRORCODE in BTEQ?

1. Definition you should say first: Branch on failure in a script. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

296. What is a return code in jobs?

1. Definition you should say first: Scheduler uses it to fail the box. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

297. How do you alert on a failed load?

1. Definition you should say first: Scheduler plus Viewpoint alerts plus mail. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

298. What is a runbook?

1. Definition you should say first: Step-by-step for a repeat incident. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

299. What is a CAB?

1. Definition you should say first: Change advisory board for production changes. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

300. What is a back-out plan?

1. Definition you should say first: How you undo a change if it fails. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Latin vs Unicode conversions cost CPU. CS comparisons surprise people. BTEQ .IF ERRORCODE drives job return codes the scheduler uses. CAB and back-out are change control.

3. Every load job: IF ERRORCODE <> 0 then .GOTO FAIL and non-zero exit. Alert Viewpoint + mail. Back-out = restore table or rerun from stg.

4. Working code / syntax (write this on Teradata SQL):

```
.LOGON td/user,pass;
DELETE FROM edw.fact_orders WHERE order_dt = DATE '2026-09-03';
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
INSERT INTO edw.fact_orders SELECT * FROM stg.orders_in;
.IF ERRORCODE <> 0 THEN .GOTO FAIL;
.QUIT 0;
.LABEL FAIL;
.QUIT 12;
```

5. A job that always exits 0 even on FastLoad fail will ship empty cubes to finance.

301. What is Tosca?

1. Definition you should say first: Tricentis Tosca: model-based test automation, largely scriptless. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tricentis Tosca is commercial model-based test automation. You scan the app into modules and reuse them; you do not start from raw Selenium code.

3. Story for interviews: Commander + modules + TCD + execution lists + DEX. Mention SAP GUI engine if that is your project.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Workspace: Customer.tws
Engines: HTML, SAP, API, Database
Run: ScratchBook locally, ExecutionList on DEX, TC-Shell in Jenkins
```

5. Calling Tosca 'scriptless so anyone can test' without naming ActionMode and identification is a shallow answer.

302. What is model-based testing in Tosca?

1. Definition you should say first: You scan the app into modules and reuse them in test cases. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. The model is the module: technical identification plus business names. Tests steer the model, so UI changes are fixed once in the module.

3. When a button id changes, rescan that attribute, not 40 test cases. Keep business names stable.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: SAP_VA01_Header
  Attribute: OrderType     Tech: GuiComboBox id=...   Business: Order Type
TestCase: Create standard order
  SAP_VA01_Header.OrderType   Input   OR
```

5. Copy-pasting modules per test destroys the model. One screen, one module family.

303. What is Tosca Commander?

1. Definition you should say first: The main UI to design, run, and manage tests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Commander is the IDE. A workspace points at a repository. Multi-user uses a DB (check-in/out). Single-user is a local file repo.

3. Enterprises run multi-user. Check out the smallest folder, change, check in with a comment. Do not keep Commander open over a VPN sleep.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Checkout: TestCases/SAP/Order
Checkin comment: TKT-1821 WaitOn busy indicator on VA01 save
Subset export: SAP_Order_2026_09.tsu
```

5. Lost checkouts block the team. Admins recover ownership; do not clone a second workspace and edit the same path.

304. What is a workspace?

1. Definition you should say first: A Tosca repository you open in Commander. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Commander is the IDE. A workspace points at a repository. Multi-user uses a DB (check-in/out). Single-user is a local file repo.

3. Enterprises run multi-user. Check out the smallest folder, change, check in with a comment. Do not keep Commander open over a VPN sleep.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Checkout: TestCases/SAP/Order
```

```
Checkin comment: TKT-1821 WaitOn busy indicator on VA01 save
Subset export: SAP_Order_2026_09.tsu
```

5. Lost checkouts block the team. Admins recover ownership; do not clone a second workspace and edit the same path.

305. Multi-user vs single-user repo?

1. Definition you should say first: Multi-user uses a DB for check-in; single-user is file-based. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Commander is the IDE. A workspace points at a repository. Multi-user uses a DB (check-in/out). Single-user is a local file repo.

3. Enterprises run multi-user. Check out the smallest folder, change, check in with a comment. Do not keep Commander open over a VPN sleep.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Checkout: TestCases/SAP/Order
Checkin comment: TKT-1821 WaitOn busy indicator on VA01 save
Subset export: SAP_Order_2026_09.tsu
```

5. Lost checkouts block the team. Admins recover ownership; do not clone a second workspace and edit the same path.

306. What is a module?

1. Definition you should say first: A reusable model of UI or API controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

307. What is XScan?

1. Definition you should say first: Scanner that identifies controls and builds modules. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

308. What is TBox?

1. Definition you should say first: Tosca's engine layer for steering applications. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

309. What is a test case?

1. Definition you should say first: A sequence of test steps using modules. Then spend the rest of the answer on

architecture, a working example, and a production failure.

2. A test case is an ordered list of module steps. Each cell is a value plus ActionMode: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert.

3. Keep one business action per folder. Put data in TCD or test configuration parameters, not hardcoded passwords.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
User.Name          Input      {CP[User]}
User.Password      Input      ****
User.Login         Input      X
Home.Title         Verify     Dashboard
Home.EmpId         Buffer     EmpId
Grid.Row           Constraint Status==Open
Grid.Status        WaitOn     Open
```

5. Wrong ActionMode is the #1 Tosca bug: Input on a label, Verify without WaitOn, Buffer overwriting a global name.

310. What is a test step value?

1. Definition you should say first: The action and data on a module attribute. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A test case is an ordered list of module steps. Each cell is a value plus ActionMode: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert.

3. Keep one business action per folder. Put data in TCD or test configuration parameters, not hardcoded passwords.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
User.Name          Input      {CP[User]}
User.Password      Input      ****
User.Login         Input      X
Home.Title         Verify     Dashboard
Home.EmpId         Buffer     EmpId
Grid.Row           Constraint Status==Open
Grid.Status        WaitOn     Open
```

5. Wrong ActionMode is the #1 Tosca bug: Input on a label, Verify without WaitOn, Buffer overwriting a global name.

311. What is ActionMode?

1. Definition you should say first: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert, etc. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A test case is an ordered list of module steps. Each cell is a value plus ActionMode: Input, Output, Verify, Buffer, WaitOn, Select, Constraint, Insert.

3. Keep one business action per folder. Put data in TCD or test configuration parameters, not hardcoded passwords.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
User.Name          Input      {CP[User]}
User.Password      Input      ****
User.Login         Input      X
Home.Title         Verify     Dashboard
Home.EmpId         Buffer     EmpId
Grid.Row           Constraint Status==Open
Grid.Status        WaitOn     Open
```

5. Wrong ActionMode is the #1 Tosca bug: Input on a label, Verify without WaitOn, Buffer overwriting a global name.

312. What is Buffer?

1. Definition you should say first: Save a runtime value to reuse later. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer      OrderId
Search.Order        Input       {B[OrderId]}
API.Body            Input       {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId     <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

313. What is Verify?

1. Definition you should say first: Assert that a control equals an expected value. Then spend the rest of the answer on

architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

314. What is WaitOn?

1. Definition you should say first: Wait until a control reaches a value or exists. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

315. What is Constraint?

1. Definition you should say first: Limit which row or item is used. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

316. What is a TestCaseDesign sheet?

1. Definition you should say first: Data-driven instances from a template. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

317. What is an instance?

1. Definition you should say first: One data combination generated from TCD. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass

values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

318. What is a template test case?

1. Definition you should say first: A test case that instantiates from TCD. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

319. What is a business parameter?

1. Definition you should say first: A named value passed into a test case. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

320. What is a library in Tosca?

1. Definition you should say first: Reusable test step folders. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

321. What is a TestStepFolder?

1. Definition you should say first: A grouping of steps, sometimes used as a reusable block. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
```

```
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

322. What is a recovery scenario?

1. Definition you should say first: Cleanup or retry after a failed step. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

323. What is a cleanup folder?

1. Definition you should say first: Steps that run after a test to leave the app stable. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

324. What is an execution list?

1. Definition you should say first: A set of test cases to run in order. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

325. What is ScratchBook?

1. Definition you should say first: Ad-hoc run without saving results to an execution list. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

326. What is TC-Shell?

1. Definition you should say first: Command-line Tosca for CI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.

3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

327. What is DEX?

1. Definition you should say first: Distributed Execution: run tests on agent machines. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DEX is Distributed Execution: a queue, agents, and Tosca Server. AOS is the Automation Object Service in newer versions.

3. One agent per machine, same SAP GUI / browser as the project. Monitor agent heartbeat. Do not RDP-lock the session if SAP needs UI.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Tosca Server URL: https://tosca.company.local
Agent: QA-DEX-01  (Win + SAP GUI 7.70 + Chrome)
Event: ExecutionList Regression_Nightly at 22:30 IST
```

5. A disconnected RDP session kills SAP GUI tests. Use a service account autologon strategy approved by IT.

328. What is a DEX agent?

1. Definition you should say first: A machine that executes tests from a queue. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DEX is Distributed Execution: a queue, agents, and Tosca Server. AOS is the Automation Object Service in newer versions.

3. One agent per machine, same SAP GUI / browser as the project. Monitor agent heartbeat. Do not RDP-lock the session if SAP needs UI.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Tosca Server URL: https://tosca.company.local
Agent: QA-DEX-01  (Win + SAP GUI 7.70 + Chrome)
Event: ExecutionList Regression_Nightly at 22:30 IST
```

5. A disconnected RDP session kills SAP GUI tests. Use a service account autologon strategy approved by IT.

329. What is Tosca Server?

1. Definition you should say first: Services for DEX, dashboards, and integrations. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tricentis Tosca is commercial model-based test automation. You scan the app into modules and reuse them; you do not start from raw Selenium code.

3. Story for interviews: Commander + modules + TCD + execution lists + DEX. Mention SAP GUI engine if that is your project.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Workspace: Customer.tws
Engines: HTML, SAP, API, Database
Run: ScratchBook locally, ExecutionList on DEX, TC-Shell in Jenkins
```

5. Calling Tosca 'scriptless so anyone can test' without naming ActionMode and identification is a shallow answer.

330. What is AOS?

1. Definition you should say first: Automation Object Service, depending on the Tosca version. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. DEX is Distributed Execution: a queue, agents, and Tosca Server. AOS is the Automation Object Service in newer versions.

3. One agent per machine, same SAP GUI / browser as the project. Monitor agent heartbeat. Do not RDP-lock the session if SAP needs UI.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Tosca Server URL: https://tosca.company.local
Agent: QA-DEX-01  (Win + SAP GUI 7.70 + Chrome)
Event: ExecutionList Regression_Nightly at 22:30 IST
```

5. A disconnected RDP session kills SAP GUI tests. Use a service account autologon strategy approved by IT.

331. What is TDS?

1. Definition you should say first: Test Data Service: managed test data. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TDS / Test Data Service provisions and masks data so tests do not steal each other's orders.

3. Create a pool of customers. Checkout a record in the test, check in after cleanup. Never hardcode a prod customer.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TDS type: Customer
  Status: Available -> InUse -> Consumed
Test: checkout Customer where Country==IN
Buffer CustId
Cleanup: set Available if the order rolled back
```

5. A shared QA client 1000 used by 12 testers will make Verify fail randomly.

332. What is Test Data Management?

1. Definition you should say first: Create, mask, and provision data for tests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TDS / Test Data Service provisions and masks data so tests do not steal each other's orders.

3. Create a pool of customers. Checkout a record in the test, check in after cleanup. Never hardcode a prod customer.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TDS type: Customer
  Status: Available -> InUse -> Consumed
Test: checkout Customer where Country==IN
Buffer CustId
Cleanup: set Available if the order rolled back
```

5. A shared QA client 1000 used by 12 testers will make Verify fail randomly.

333. How do you scan a web page?

1. Definition you should say first: XScan with the browser extension or engine, then map controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

334. What is a steering engine?

1. Definition you should say first: The adapter for HTML, SAP, API, and others. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

335. What is the HTML engine?

1. Definition you should say first: Steers browser DOM controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

336. What is the SAP engine?

1. Definition you should say first: Steers SAP GUI controls. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

337. What is API Scan?

1. Definition you should say first: Build modules from REST/SOAP definitions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

338. How do you test REST in Tosca?

1. Definition you should say first: API Scan or API Engine modules with payload and verify. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

339. What is a payload?

1. Definition you should say first: JSON or XML body you send. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

340. How do you verify status 200?

1. Definition you should say first: Verify the status attribute on the API module. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

341. How do you chain API calls?

1. Definition you should say first: Buffer an id from the first response into the second request. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. API Scan builds modules from OpenAPI/WSDL. Send method, headers, body. Verify status and JSON. Buffer ids to chain calls.

3. Do create via API then Verify in UI, or the reverse. Keep secrets in TCPs.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
POST /api/orders
Headers: Authorization = Bearer {CP[Token]}
Body: {"material":"MAT1","qty":1}
Status        Verify    201
Json.id       Buffer    OrderId
GET /api/orders/{B[OrderId]}
Status        Verify    200
Json.status   Verify    OPEN
```

5. Verifying only 200 on a GET that returned an error HTML page is a false green. Verify a business field.

342. What is a module attribute?

1. Definition you should say first: One field or control in a module. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
```

```
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

343. What is a technical ID?

1. Definition you should say first: How Tosca finds the control, for example id or xpath-like path. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

344. What if XScan cannot find a control?

1. Definition you should say first: Try another engine, frame, or identify by parent. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A module is a reusable control tree. XScan builds it. TBox engines steer HTML, SAP, API, etc. Each attribute has a technical ID plus a business name.

3. Scan only the controls you need. Disable volatile ids. Add a parent constraint for tables. Rescan when the screen changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
XScan -> select controls -> map names
Technical: id=submitApp  OR  xpath-like path  OR  SAP name=...
Business: Submit
Engine: Html or Sap
```

5. Dynamic ids (ctl00_...) break after every deploy. Switch to a stable property or an anchor parent.

345. What is a dynamic ID problem?

1. Definition you should say first: IDs change each session; use a stable property. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

346. How do you handle iFrames?

1. Definition you should say first: Scan inside the frame or switch context per engine rules. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

347. How do you handle a dropdown?

1. Definition you should say first: Input or Select action on the combo control. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

348. How do you handle a table?

1. Definition you should say first: Constraint on a row plus Input/Verify on cells. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

349. How do you upload a file?

1. Definition you should say first: File control Input with a path the agent can see. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

350. How do you handle CAPTCHA?

1. Definition you should say first: You do not automate CAPTCHA; tests skip or use a test hook. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

351. How do you handle SSO?

1. Definition you should say first: Test account plus storage of a session if the project allows. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Web: XScan HTML engine, watch frames, stable ids, table Constraint, file path the agent can see. CAPTCHA is not automated. SSO uses a test IdP or a stored session if security allows.

3. For dynamic ids identify by label text or a parent container. For iframes scan inside the frame.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Frame: PaymentFrame
  CardNumber  Input  4242...
Table.Row    Constraint  SKU=={B[Sku]}
Table.Qty    Input       2
File.Upload  Input       C:\agents\data\resume.pdf
# CAPTCHA: skip / test hook, never production bypass
```

5. Uploading a path that exists only on your laptop fails on DEX. Put fixtures on the agent or a share.

352. What is a business-focused test case?

1. Definition you should say first: Steps named in business language, modules underneath. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

353. What is risk-based testing in Tosca?

1. Definition you should say first: Weight tests by business risk if the project uses it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

354. What is exploratory testing vs Tosca?

1. Definition you should say first: Exploratory is unscripted; Tosca is designed automation. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

355. What is a requirements folder?

1. Definition you should say first: Optional mapping of tests to requirements. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.

3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

356. What is coverage in Tosca?

1. Definition you should say first: Which requirements or TCD instances ran. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Business-focused cases use business language. Risk-based testing weights high-impact paths. Exploratory is unscripted and complements Tosca. Requirements folders map coverage.
3. Link smoke cases to login and order-create requirements. Show coverage in Commander, not a spreadsheet only.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Requirement: R-Order-Create
  TestCases: TC_VA01_Happy, TC_VA01_RejectQty
Coverage: 2/2 executed on ExecutionList Release_54
```

5. 100% instance coverage of a low-risk screen and 0% on billing is fake quality.

357. How do you run from CI?

1. Definition you should say first: TC-Shell or Tosca CI against a workspace and execution list. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

358. How do you pass results to Jenkins?

1. Definition you should say first: CI result files or Tosca Server reporting. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

359. What is a configuration parameter?

1. Definition you should say first: Settings such as browser, timeout, or test config. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.
3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.
4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

360. What is TCP?

1. Definition you should say first: Test configuration parameter. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.

3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

361. How do you run Chrome vs Edge?

1. Definition you should say first: TCP or test configuration for the browser. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TC-Shell is the CLI. CI checks out the workspace, runs an execution list, publishes results. TCPs set browser, URL, user, timeout.

3. Keep credentials in Jenkins credentials, not in the repo. Pin browser TCP to the agent image.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCShell.exe -workspace C:\Tosca\Proj.tws -executionlist "Smoke_SAP_R3"
TCP Browser: Chrome
TCP Url: https://qa.example.com
TCP Timeout: 20000
Exit code != 0 fails the Jenkins stage
```

5. Agents without the matching browser pack or Tosca license silently skip engines.

362. What is a buffer syntax?

1. Definition you should say first: Curly braces with the buffer name, depending on version. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer     OrderId
Search.Order        Input      {B[OrderId]}
API.Body            Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

363. What is a date expression in Tosca?

1. Definition you should say first: Dynamic date functions in test step values. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca date expressions compute today+N. Random values need a stored seed if the value must be reused or cleaned up.

3. Use {DATE[yyyyMMdd][+1]} style expressions per your version. Buffer random keys immediately.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ValidFrom      Input   {DATE[dd.MM.yyyy]}
ValidTo        Input   {DATE[dd.MM.yyyy][+30]}
PoNumber       Input   PO{RANDOM[6]}
PoNumber       Buffer  PoNumber
```

5. Random emails without a unique domain will collide and poison the environment.

364. What is a random value?

1. Definition you should say first: Generated test data; do not use it for unique keys without control. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca date expressions compute today+N. Random values need a stored seed if the value must be reused or cleaned up.

3. Use {DATE[yyyyMMdd][+1]} style expressions per your version. Buffer random keys immediately.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ValidFrom      Input   {DATE[dd.MM.yyyy]}
ValidTo        Input   {DATE[dd.MM.yyyy][+30]}
PoNumber       Input   PO{RANDOM[6]}
PoNumber       Buffer  PoNumber
```

5. Random emails without a unique domain will collide and poison the environment.

365. How do you wait for a spinner?

1. Definition you should say first: WaitOn existence or wait for the spinner to vanish. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

366. Why not use a huge static wait?

1. Definition you should say first: It slows suites and still flakes; WaitOn is better. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

367. What is synchronization?

1. Definition you should say first: Waiting for the app to be ready before the next step. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

368. What is a flaky test?

1. Definition you should say first: Passes and fails without product bugs; fix waits and identifications. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Verify asserts. WaitOn waits until a property matches. Constraint picks a row. Synchronization is WaitOn, not a 20s pause.

3. WaitOn busy=false or existence of the next screen, then Verify. For spinners wait until the spinner vanishes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.Busy            WaitOn     false
Save.Message        WaitOn     Saved*
Save.Message        Verify     Saved successfully
Table.Row           Constraint Order=={B[OrderId]}
Table.Status        Verify     Released
# do not: Wait 30000 on every step
```

5. Huge static waits slow DEX and still flake on a slow AMP-side batch. Interviewers will ask what you WaitOn.

369. How do you stabilize SAP tests?

1. Definition you should say first: SAP engine, proper GUI scripting, and wait for busy indicators. Then spend the rest of the

answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

370. What is SAP scripting setting?

1. Definition you should say first: SAP GUI must allow scripting on the client and server. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

371. What is a TCode?

1. Definition you should say first: SAP transaction code you steer to. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

372. How do you test an SAP Fiori app?

1. Definition you should say first: Often HTML/browser engine, not classic SAP GUI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Classic SAP GUI needs scripting enabled on client and server. Fiori is HTML/Fiori engine, not GuiButton. TCode is the transaction (VA01).

3. WaitOn SAP busy. Use /nTCODE to jump. After fail, recovery to SAP Easy Access. Match GUI patch on all agents.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
SAP.OkCode          Input     /nVA01
SAP.Busy            WaitOn    false
VA01.OrderType      Input     OR
VA01.SalesOrg       Input     1000
VA01.Save           Input     X
VA01.StatusBar      Verify    was saved
# saplogon.ini + scripting = enabled
```

5. If scripting is off, Tosca sees a dead GUI. This is an infra check, not a module problem.

373. What is a Tosca license type?

1. Definition you should say first: Commander, execution-only, etc. - follow what the project bought. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

374. What is a subset?

1. Definition you should say first: An export of part of the repo to share. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

375. How do you avoid merge conflicts?

1. Definition you should say first: Small check-outs, folders by team, frequent check-in. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca is model-based: XScan builds modules, test cases steer them, TCD supplies data, DEX runs agents.

3. Name ActionMode, buffers, WaitOn, and how CI calls TC-Shell. Never demo a 30-second static wait.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Login.User          Input     {CP[User]}
Login.Password      Input     {PW}
Login.SignIn        Input     X
Home.Welcome        Verify    Welcome*
Home.OrderId        Buffer    OrderId
```

5. Flakes come from dynamic IDs, missing SAP scripting, shared buffers, and no recovery after a leftover dialog.

376. What is a checkout?

1. Definition you should say first: Lock objects so you can edit in multi-user. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

377. What is a check-in?

1. Definition you should say first: Save objects back to the common repo. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

378. What happens if you lose a checkout?

1. Definition you should say first: Admin recover or take ownership per process. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

379. What is a branch in Tosca?

1. Definition you should say first: Versioning approach depends on repo type and version. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Licenses: Commander vs execution-only. Subsets share objects. Checkout locks. Check-in publishes. Lost checkout needs admin take-ownership. Branching depends on version (Git integration vs repo copies).

3. Small folders, daily check-in, one owner per module family.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
toscaclient checkout /path/Modules/SAP/VA01
# edit
toscaclient checkin -m "TKT-44 add WaitOn on save"
```

5. Editing in a personal subset and never merging creates shadow modules.

380. How do you review a test case?

1. Definition you should say first: Peer review modules, data, and verifies, not only a green run. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

381. What is a smoke execution list?

1. Definition you should say first: A short list that proves the app starts and login works. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

382. What is a regression pack?

1. Definition you should say first: The suite you run before a release. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
```

```
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

383. How do you select tests for a change?

1. Definition you should say first: Impact on modules used by the changed screens. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Execution lists are the runnable suite. ScratchBook is a scratch run that does not store official results. Smoke is login plus one create. Regression is the release pack.

3. Map modules touched by a change to tests that use them. Do not run 2000 cases for a label color change.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
ExecutionList: Smoke_SAP_R3
  01_Login
  02_VA01_CreateOrder
  03_Logout
CI: TC-Shell executes this list after every build
```

5. Green ScratchBook and red DEX usually means agent path, browser, or SAP scripting differ from your laptop.

384. What is a data-driven test?

1. Definition you should say first: Same steps, many TCD instances. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

385. What is a keyword-driven test?

1. Definition you should say first: Reusable business steps; Tosca libraries play that role. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. TCD is a sheet of attributes and instances. A template test instantiates one case per instance. Business parameters pass values in. Libraries are keyword-like reusable folders.

3. Put equivalence classes in TCD (valid, min, max, reject). Link the template. Instantiate before the run.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
TCD Sheet OrderCreate
  Attribute: OrderType  | Customer | Qty
  Instance Happy: OR | 1000 | 1
  Instance Reject: XX | 1000 | 0
Template uses {OrderType} {Customer} {Qty}
```

5. 500 instances with no risk class is noise. Tag smoke vs regression instances.

386. Tosca vs Selenium?

1. Definition you should say first: Tosca is model-based commercial; Selenium is code-first OSS. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

387. When is Tosca a poor fit?

1. Definition you should say first: Tiny apps, heavy CAPTCHA, or teams that only want code. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

388. What is a TestCase workstate?

1. Definition you should say first: Status such as COMPLETED used in some processes. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

389. What is a log in Tosca?

1. Definition you should say first: Run log with passed/failed steps and screenshots if configured. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

390. How do you capture a screenshot on fail?

1. Definition you should say first: Execution settings for screenshots. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

391. What is a recovery scenario example?

1. Definition you should say first: Close a leftover dialog so the next case can start. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
```

```
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

392. How do you skip a step?

1. Definition you should say first: Condition or disable the step; do not leave broken verifies. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

393. What is a condition in Tosca?

1. Definition you should say first: If-then style control of whether steps run. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

394. What is a repetition?

1. Definition you should say first: Loop a folder while a condition holds. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

395. How do you loop a table?

1. Definition you should say first: Repetition with Constraint moving through rows. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Libraries and TestStepFolders reuse steps. Recovery runs after a failure (close a modal). Cleanup always runs. Conditions skip. Repetition loops with Constraint advancing rows.

3. Design recovery to leave SAP at the Easy Access menu. Loop with a clear exit (no more rows).

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
IF {B[SkipAddr]} == Yes -> disable Address folder
Repetition: Table.Row Constraint  (next unmatched)
  Table.Cell  Verify  {Expected}
Recovery: Popup.Close  Input X
Cleanup: SAP./n  Input  /n   (back to menu)
```

5. Infinite repetitions without a max count hang DEX agents.

396. What is a buffer overwrite risk?

1. Definition you should say first: Two tests share a buffer name and leak data. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer     OrderId
Search.Order        Input      {B[OrderId]}
API.Body            Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

397. How do you isolate buffers?

1. Definition you should say first: Unique names per test or clear them in cleanup. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Buffer stores a runtime value. Later steps read {B[Name]}. Buffers are workspace-session scoped unless you isolate names.

3. Buffer ids immediately after create. Prefix with test name. Clear in cleanup. Prefer business parameters for input data.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Order.Number        Buffer     OrderId
Search.Order        Input      {B[OrderId]}
API.Body            Input      {"id":"{B[OrderId]}"}
Cleanup: Buffer     OrderId    <empty>   ActionMode: Input (reset)
```

5. Two parallel DEX agents writing OrderId will leak data across tests. Unique names or TDS.

398. What is a test configuration 'Standard'?

1. Definition you should say first: Default config used if none is specified. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Standard is the default TCP set when none is selected. Agents and laptops must agree on URL and browser or results lie.

3. Create QA, UAT, PROD-readonly configs. Never point Standard at production.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Config Standard: Url=https://qa.example.com Browser=Chrome
Config UAT: Url=https://uat.example.com Browser=Edge
```

5. A leftover Standard URL from a demo will smash the wrong environment.

399. How do you handle 2FA in tests?

1. Definition you should say first: Test hooks or a non-prod OTP process. Never bypass production 2FA. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

400. Should you automate production?

1. Definition you should say first: Prefer test environments; production automation is rare and gated. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

401. What is a service virtualization?

1. Definition you should say first: Fake a dependency; Tosca may call a stub instead of a live API. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

402. What is a contract test?

1. Definition you should say first: API schema and status checks between services. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

403. How do you version modules?

1. Definition you should say first: Change with the app; keep old tests green or retire them. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

404. What is technical debt in Tosca?

1. Definition you should say first: Duplicate modules, absolute waits, and unused instances. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

405. How do you name modules?

1. Definition you should say first: Screen plus control group, stable and unique. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
```

```
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

406. How do you name test cases?

1. Definition you should say first: Story or risk plus expected outcome. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Tosca wins on SAP GUI and model reuse. Selenium/Playwright win on code-centric web teams. Do not automate prod, CAPTCHA, or real 2FA. Contract tests belong at API. Virtualize unstable deps.

3. Name modules Screen_ControlGroup. Name cases Story_ExpectedResult. Rescan, do not clone, when UI changes.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Module: Web_Apply_Personal
TestCase: APPLY-12_NoticePeriod_persists_after_save
2FA: test OTP hook on QA only
Prod: no unattended submit
```

5. Debt: duplicate modules, 30s waits, unused TCD instances, passwords in step values.

407. What is a Definition of Done for a test?

1. Definition you should say first: Runs green twice on CI, reviewed, and data is documented. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Review modules, identification, WaitOn, data, and verifies - not only a green ScratchBook. Logs and screenshots on fail live in the execution result.

3. DoD: two green DEX runs, reviewed, data source documented, recovery present.

4. Working code / syntax (write this on Tosca Commander / TC-Shell):

```
Execution result: Failed step Home.Save Verify
Expected: Saved*
Actual: Busy
Screenshot: 2026-09-03_12-11-02.png
```

5. A test that is COMPLETED in workstate but never ran on DEX is not done.

408. What is Playwright?

1. Definition you should say first: A Microsoft library to automate Chromium, Firefox, and WebKit. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Playwright is Microsoft's open-source library for reliable end-to-end tests. One API covers Chromium, Firefox, and WebKit, with browsers installed via `npx playwright install`.

3. In interviews, contrast it with Selenium: auto-wait, traces, API mocking, and test fixtures are built in. Node/TS is the default at most web teams; Python uses the same browser engine.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npm init playwright@latest
npx playwright install
npx playwright test --project=chromium
npx playwright show-report
```

5. If you only say 'it automates browsers' you fail the next question. Name locators, contexts, traces, and storageState.

409. Playwright vs Selenium?

1. Definition you should say first: Auto-wait, browsers included, and a single API across engines. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Selenium talks WebDriver and often needs explicit waits. Playwright talks browser protocols, auto-waits for actionability, and ships browsers.

3. Migration talk: map drivers to browserType, By.css to getByRole, and TestNG suites to playwright.config projects. Keep business assertions; throw away Thread.sleep.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('textbox', { name: 'Email' }).fill('user@test.com');
await expect(page.getByText('Signed in')).toBeVisible();
// Selenium equivalent needed WebDriverWait + ExpectedConditions
```

5. Do not trash Selenium. Say where it still wins: Appium mobile, old IE, or a huge existing grid.

410. What is a browser context?

1. Definition you should say first: An isolated incognito-like session inside a browser. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A context is an isolated incognito-like profile: cookies, storage, permissions. One browser can host many contexts.
3. Playwright Test gives you a fresh context per test by default. Use storageState to inject a saved login into that context.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
const context = await browser.newContext({
  storageState: 'playwright/.auth/user.json',
  locale: 'en-IN',
  viewport: { width: 1280, height: 720 },
});
const page = await context.newPage();
```
5. Sharing one context across tests leaks cookies and makes parallel runs flaky.

411. What is a page?

1. Definition you should say first: A tab inside a context. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A page is a tab. Almost every user action (goto, click, fill) hangs off page or a locator created from page.
3. Wait for popup pages when a click opens a tab. Never assume page.locator after a navigation without an assertion.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Open report' }).click(),
]);
await popup.waitForLoadState();
await expect(popup).toHaveURL(/report/);
```
5. Holding a page after context close throws. Isolate pages per test.

412. What is a locator?

1. Definition you should say first: A lazy handle to find elements, not a one-time query. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A locator is a lazy query that re-runs until the action succeeds. An ElementHandle is a snapshot that goes stale after rerender.
3. Create locators with getByRole / getByTestId / getByLabel. Chain filter and nth only after the role is correct.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
const save = page.getByRole('button', { name: 'Save' });
await save.click();
await expect(save).toBeEnabled();
// bad: const h = await page.$('#save'); await h.click();
```
5. If you cache handles in a POM constructor, React rerenders will flake. Store locators, not handles.

413. Why are locators better than element handles?

1. Definition you should say first: They re-query and wait; handles go stale. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A locator is a lazy query that re-runs until the action succeeds. An ElementHandle is a snapshot that goes stale after rerender.
3. Create locators with getByRole / getByTestId / getByLabel. Chain filter and nth only after the role is correct.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
const save = page.getByRole('button', { name: 'Save' });
await save.click();
await expect(save).toBeEnabled();
// bad: const h = await page.$('#save'); await h.click();
```
5. If you cache handles in a POM constructor, React rerenders will flake. Store locators, not handles.

414. What is getByRole?

1. Definition you should say first: Locate by accessible role and name. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. getByRole uses the accessibility tree: role plus accessible name. This matches how keyboard and screen-reader users see the page.
3. Use name as a string or regex. Add exact: true when two buttons share a prefix. Prefer this over CSS ids.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
await page.getByRole('button', { name: 'Submit application' }).click();
await page.getByRole('textbox', { name: 'Email' }).fill('a@b.com');
await page.getByRole('combobox', { name: 'Country' }).selectOption('IN');
```

5. Icon-only buttons without an accessible name cannot be found. Fix the app (aria-label) or fall back to test id.

415. What is getByTestId?

1. Definition you should say first: Locate by a test id attribute you control. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. getByTestId targets a stable hook you own, usually data-testid. Use it when roles collide or the UI is canvas-like.

3. Agree the attribute in playwright.config testIdAttribute. Do not sprinkle test ids on every div; prefer roles first.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
// playwright.config.ts
export default { use: { testIdAttribute: 'data-testid' } };
await page.getByTestId('order-total').click();
await expect(page.getByTestId('order-total')).toHaveText('INR 1,200');
```

5. If frontend renames test ids without telling QA, CI goes red. Treat them as a contract.

416. What is getByText?

1. Definition you should say first: Locate by visible text. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. getByText finds visible text nodes. Good for messages; bad for buttons (use getByRole) and for text that is translated.

3. Pass exact or regex. Combine with a parent locator so you do not hit a footer duplicate.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page.getByText('Application submitted', { exact: true })).toBeVisible();
await page.locator('main').getByText(/saved at/i).click();
```

5. Marketing copy changes weekly. Prefer role+name or test id for actions.

417. What is a strict mode violation?

1. Definition you should say first: Locator resolved to more than one element. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Strict mode fails when a locator resolves to more than one element. That is a feature: ambiguous locators hide bugs.

3. Tighten with getByRole name, filter({ hasText }), or nth(0) only when the list is intentional.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const row = page.getByRole('row').filter({ hasText: 'ORD-1042' });
await row.getByRole('button', { name: 'Edit' }).click();
await page.getByRole('listitem').nth(2).click();
```

5. nth(0) on a duplicated Submit button is a smell. Fix the locator or the DOM.

418. How do you pick one of many?

1. Definition you should say first: nth, filter, or a tighter locator. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Strict mode fails when a locator resolves to more than one element. That is a feature: ambiguous locators hide bugs.

3. Tighten with getByRole name, filter({ hasText }), or nth(0) only when the list is intentional.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const row = page.getByRole('row').filter({ hasText: 'ORD-1042' });
await row.getByRole('button', { name: 'Edit' }).click();
await page.getByRole('listitem').nth(2).click();
```

5. nth(0) on a duplicated Submit button is a smell. Fix the locator or the DOM.

419. What is auto-waiting?

1. Definition you should say first: Playwright waits for actionability before click or fill. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Before click/fill, Playwright waits until the element is visible, enabled, stable, and receiving events.

3. You rarely need waitForSelector. If an action times out, inspect intercepting overlays and disabled states in the trace.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Pay' }).click({ timeout: 15000 });
// waits for actionability, then clicks
```

5. force: true skips actionability and causes 'it clicked in CI but UI did nothing' bugs.

420. What is actionability?

1. Definition you should say first: Visible, enabled, stable, and receiving events. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Before click/fill, Playwright waits until the element is visible, enabled, stable, and receiving events.

3. You rarely need waitForSelector. If an action times out, inspect intercepting overlays and disabled states in the trace.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Pay' }).click({ timeout: 15000 });
// waits for actionability, then clicks
```

5. force: true skips actionability and causes 'it clicked in CI but UI did nothing' bugs.

421. What is a timeout in Playwright?

1. Definition you should say first: Default 30s for many actions; set per test or per action. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Default action and expect timeout is 30s (configurable). Test timeout is separate (often 30s-60s per test).
3. Set a realistic test timeout in config. Override per slow navigation. Never set 0 globally.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export default defineConfig({
  timeout: 60_000,
  expect: { timeout: 10_000 },
  use: { actionTimeout: 15_000, navigationTimeout: 30_000 },
});
```

5. A 5-minute timeout hides deadlocks. Fail fast, then add a targeted wait for a known spinner.

422. What is expect(locator).toBeVisible()?

1. Definition you should say first: An assertion that waits until the element is visible. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web-first assertions retry until they pass or time out. expect(locator).toBeVisible() is not a one-shot check.
3. Assert outcomes, not sleeps. Chain toHaveURL, toHaveText, toHaveCount after the action.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page.getByRole('dialog')).toBeVisible();
await expect(page.getByRole('dialog')).toContainText('Confirm delete');
await expect(page.getByRole('dialog')).toBeHidden();
```

5. Non-retrying expect(await locator.textContent()).toBe() races the UI. Use locator assertions.

423. Web-first assertions?

1. Definition you should say first: Assertions that retry until they pass or time out. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Web-first assertions retry until they pass or time out. expect(locator).toBeVisible() is not a one-shot check.
3. Assert outcomes, not sleeps. Chain toHaveURL, toHaveText, toHaveCount after the action.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page.getByRole('dialog')).toBeVisible();
await expect(page.getByRole('dialog')).toContainText('Confirm delete');
await expect(page.getByRole('dialog')).toBeHidden();
```

5. Non-retrying expect(await locator.textContent()).toBe() races the UI. Use locator assertions.

424. What is a fixture?

1. Definition you should say first: Shared setup in Playwright Test, such as page. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Fixtures inject setup (page, logged-in page, API client). test.extend adds your own.
3. Put login, seed, and teardown in fixtures so tests stay one screen of business steps.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
export const test = base.extend< { adminPage: Page } >({
  adminPage: async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'playwright/.auth/admin.json' });
    const page = await ctx.newPage();
    await use(page);
    await ctx.close();
  },
});
```

5. Heavy fixture work in every test without worker reuse will explode CI time.

425. What is test.extend?

1. Definition you should say first: Custom fixtures for login or data. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Fixtures inject setup (page, logged-in page, API client). test.extend adds your own.

3. Put login, seed, and teardown in fixtures so tests stay one screen of business steps.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export const test = base.extend< { adminPage: Page } >({
  adminPage: async ({ browser }, use) => {
    const ctx = await browser.newContext({ storageState: 'playwright/.auth/admin.json' });
    const page = await ctx.newPage();
    await use(page);
    await ctx.close();
  },
});
```

5. Heavy fixture work in every test without worker reuse will explode CI time.

426. What is storageState?

1. Definition you should say first: Saved cookies/localStorage to skip login. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. storageState is a JSON dump of cookies and localStorage. A setup project logs in once and writes it.

3. Mark setup as a dependency of chromium/firefox projects. Never login with UI in every test unless you are testing login itself.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
setupProject: { name: 'setup', testMatch: /auth\.setup\.ts/ }
await page.context().storageState({ path: 'playwright/.auth/user.json' });
use: { storageState: 'playwright/.auth/user.json' }
```

5. Do not commit real SSO cookies. Use a test IdP and gitignore the auth folder if it has secrets.

427. How do you reuse login?

1. Definition you should say first: Save storageState in a setup project. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. storageState is a JSON dump of cookies and localStorage. A setup project logs in once and writes it.

3. Mark setup as a dependency of chromium/firefox projects. Never login with UI in every test unless you are testing login itself.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
setupProject: { name: 'setup', testMatch: /auth\.setup\.ts/ }
await page.context().storageState({ path: 'playwright/.auth/user.json' });
use: { storageState: 'playwright/.auth/user.json' }
```

5. Do not commit real SSO cookies. Use a test IdP and gitignore the auth folder if it has secrets.

428. What is a project in playwright.config?

1. Definition you should say first: A browser or setup target. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A project is a named target: browser, setup, or device. Workers are OS processes. fullyParallel runs tests in a file together.

3. Split smoke vs regression with grep or testIgnore. Keep worker count near CPU count in CI.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
]
fullyParallel: true
workers: process.env.CI ? 2 : undefined
```

5. Parallel tests that write the same user row or the same download folder will collide.

429. What is a worker?

1. Definition you should say first: A parallel process that runs tests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A project is a named target: browser, setup, or device. Workers are OS processes. fullyParallel runs tests in a file together.

3. Split smoke vs regression with grep or testIgnore. Keep worker count near CPU count in CI.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
```

```
]
fullyParallel: true
workers: process.env.CI ? 2 : undefined
```
5. Parallel tests that write the same user row or the same download folder will collide.

430. What is fullyParallel?

1. Definition you should say first: Run tests in a file in parallel. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A project is a named target: browser, setup, or device. Workers are OS processes. fullyParallel runs tests in a file together.
3. Split smoke vs regression with grep or testIgnore. Keep worker count near CPU count in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'] }, dependencies: ['setup'] },
]
fullyParallel: true
workers: process.env.CI ? 2 : undefined
```
5. Parallel tests that write the same user row or the same download folder will collide.

431. How do you stop test pollution?

1. Definition you should say first: Isolated contexts and no shared mutable files. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Pollution is leftover state: cookies, users, files, clocks. Isolation is a context per test plus unique data.
3. Create data via API with a random suffix. Clean up in fixture teardown if the env is shared.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
const email = `qa.${Date.now()}@mail.test`;
await request.post('/api/users', { data: { email, role: 'applicant' } });
```
5. A passed test that leaves an open dialog can fail the next test in headed debug only - still pollution.

432. What is test.step?

1. Definition you should say first: A named chunk in the report. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. test.step names a chunk in the HTML report and trace. Use it for login, fill, assert blocks.
3. Keep steps business-shaped: 'Submit ATS form', not 'click div 3'.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
await test.step('submit form', async () => {
  await page.getByLabel('Notice period').fill('30');
  await page.getByRole('button', { name: 'Continue' }).click();
});
```
5. Nested steps 8 levels deep make traces unreadable.

433. What is a trace?

1. Definition you should say first: A recording you can open in Trace Viewer. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A trace is a zip of DOM snapshots, screenshots, network, and console. Trace Viewer time-travels the run.
3. In CI keep traces on first retry or on failure to save disk. Locally use --trace on.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
npx playwright show-trace test-results/trace.zip
```
5. Always-on traces in a 2000-test nightly will fill the agent disk.

434. When do you keep traces?

1. Definition you should say first: On first retry or on failure in CI. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. A trace is a zip of DOM snapshots, screenshots, network, and console. Trace Viewer time-travels the run.
3. In CI keep traces on first retry or on failure to save disk. Locally use --trace on.
4. Working code / syntax (write this on Playwright (TypeScript)):
```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
npx playwright show-trace test-results/trace.zip
```
5. Always-on traces in a 2000-test nightly will fill the agent disk.

435. What is Trace Viewer?

1. Definition you should say first: UI to inspect DOM, network, and actions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A trace is a zip of DOM snapshots, screenshots, network, and console. Trace Viewer time-travels the run.

3. In CI keep traces on first retry or on failure to save disk. Locally use --trace on.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
use: { trace: 'on-first-retry', screenshot: 'only-on-failure', video: 'retain-on-failure' }
npx playwright show-trace test-results/trace.zip
```

5. Always-on traces in a 2000-test nightly will fill the agent disk.

436. What is codegen?

1. Definition you should say first: Record actions into a test script. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Codegen records clicks into a script. It is a start, not a suite.

3. Re-record, then replace CSS with roles, add assertions, extract POM/fixtures, delete waits.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright codegen https://careers.example.com
# then rewrite:
await page.getByRole('textbox', { name: 'Phone' }).fill('9999999999');
```

5. Shipping raw codegen is a common junior red flag: brittle selectors and no assertions.

437. Why not ship raw codegen?

1. Definition you should say first: It is a start; you still clean locators and asserts. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Codegen records clicks into a script. It is a start, not a suite.

3. Re-record, then replace CSS with roles, add assertions, extract POM/fixtures, delete waits.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright codegen https://careers.example.com
# then rewrite:
await page.getByRole('textbox', { name: 'Phone' }).fill('9999999999');
```

5. Shipping raw codegen is a common junior red flag: brittle selectors and no assertions.

438. What is page.route?

1. Definition you should say first: Intercept and mock network requests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. page.route intercepts requests. fulfill returns a stub body so UI tests do not hit real backends.

3. Mock only the unstable dependency. Let the app static assets load normally.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route('**/api/jobs*', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ jobs: [{ id: 'J1', title: 'Tosca Engineer' }] }),
  });
});
```

5. A mock that never expires will hide contract breaks. Pair with one real API smoke test.

439. How do you mock an API?

1. Definition you should say first: route fulfill with JSON. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. page.route intercepts requests. fulfill returns a stub body so UI tests do not hit real backends.

3. Mock only the unstable dependency. Let the app static assets load normally.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route('**/api/jobs*', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ jobs: [{ id: 'J1', title: 'Tosca Engineer' }] }),
  });
});
```

5. A mock that never expires will hide contract breaks. Pair with one real API smoke test.

440. How do you wait for a response?

1. Definition you should say first: page.waitForResponse with a URL predicate. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. waitForResponse ties the click to the network you care about, better than waitForLoadState('networkidle').
3. Start waiting before the click. Assert status and JSON.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [resp] = await Promise.all([
  page.waitForResponse((r) => r.url().includes('/apply') && r.status() === 201),
  page.getByRole('button', { name: 'Submit' }).click(),
]);
expect((await resp.json()).id).toBeTruthy();
```

5. Matching the wrong URL (analytics) makes the test pass while the save failed.

441. What is APIRequestContext?

1. Definition you should say first: HTTP requests without a browser, for setup or API tests. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. APIRequestContext is HTTP without a browser. Use it to seed, login, and assert APIs.
3. Create the applicant via API, then open UI to verify the list. Faster and less flake.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
test('list shows seeded job', async ({ request, page }) => {
  await request.post('/api/jobs', { data: { title: 'Playwright QA', ctc: 30 } });
  await page.goto('/jobs');
  await expect(page.getByRole('cell', { name: 'Playwright QA' })).toBeVisible();
});
```

5. Do not put production bearer tokens in repo. Use env and a test tenant.

442. How do you test downloads?

1. Definition you should say first: Wait for download event and save the file. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Wait for the download event, then save to a unique path. Assert suggestedFilename and file bytes if needed.
3. Enable acceptDownloads (default in Playwright Test). Clean the folder in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export PDF' }).click(),
]);
expect(download.suggestedFilename()).toMatch(/interview-tosca\.pdf/);
await download.saveAs('test-results/tosca.pdf');
```

5. A shared Downloads directory across workers overwrites files.

443. How do you test uploads?

1. Definition you should say first: setInputFiles on the file input. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. setInputFiles works on <input type=file>, including hidden inputs. Chrome extensions cannot do this; Playwright tests can.
3. Pass a path Playwright can read on the agent. For multiple files, pass an array.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Resume').setInputFiles('fixtures/RaviKumar.pdf');
await page.locator('input[type=file]').setInputFiles(['a.pdf', 'b.pdf']);
```

5. Custom dropzones that ignore the input need DataTransfer dispatch. Prefer asking devs for an input.

444. Can Playwright set a file input?

1. Definition you should say first: Yes in tests; a Chrome extension cannot. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. setInputFiles works on <input type=file>, including hidden inputs. Chrome extensions cannot do this; Playwright tests can.
3. Pass a path Playwright can read on the agent. For multiple files, pass an array.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Resume').setInputFiles('fixtures/RaviKumar.pdf');
await page.locator('input[type=file]').setInputFiles(['a.pdf', 'b.pdf']);
```

5. Custom dropzones that ignore the input need DataTransfer dispatch. Prefer asking devs for an input.

445. How do you handle a new tab?

1. Definition you should say first: Wait for a popup page event. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Clicks that open a tab fire a popup event. Wait for it before talking to the new page.
3. Close extra pages in teardown so workers do not leak.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [tab] = await Promise.all([
  page.waitForEvent('popup'),
  page.getByRole('link', { name: 'Privacy' }).click(),
]);
await expect(tab).toHaveTitle(/Privacy/);
```

5. If the site uses target=_blank plus noopener, you still get a Page object via the event.

446. How do you handle a dialog?

1. Definition you should say first: page.on('dialog') accept or dismiss. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Native alert/confirm/prompt are dialog events. Register the listener before the action.
3. Accept or dismiss explicitly. Unhandled dialogs stall the test.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
page.once('dialog', async (d) => {
  expect(d.message()).toContain('Delete');
  await d.accept();
});
await page.getByRole('button', { name: 'Delete' }).click();
```

5. A leftover listener from a previous test can auto-accept a later dialog.

447. How do you run headed on a laptop?

1. Definition you should say first: headed: true or --headed. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Headed is for local debug. CI should be headless, with retries 1-2, HTML report, and traces on failure.
3. Use xvfb only if a dependency needs a display. GitHub Actions ubuntu works headless.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test --headed
npx playwright test --project=chromium --reporter=html
# CI: retries: 2, workers: 2, forbid only
```

5. Do not debug only headed; some timing bugs appear only headless.

448. How do you run in CI?

1. Definition you should say first: headless, HTML report, retries 1-2. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Headed is for local debug. CI should be headless, with retries 1-2, HTML report, and traces on failure.
3. Use xvfb only if a dependency needs a display. GitHub Actions ubuntu works headless.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test --headed
npx playwright test --project=chromium --reporter=html
# CI: retries: 2, workers: 2, forbid only
```

5. Do not debug only headed; some timing bugs appear only headless.

449. What is a flake retry?

1. Definition you should say first: Re-run a failed test; fix the flake, do not hide it forever. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Retries re-run a failed test. Screenshots/videos on failure give evidence. Retries are a safety net, not a fix.
3. Quarantine a flake with test.fixme or a bug ticket. Track retry rate in CI.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
retries: process.env.CI ? 2 : 0
use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

5. Three retries on a 10% flake still fail the nightly and waste 3x minutes.

450. What is a screenshot on failure?

1. Definition you should say first: Configured in use: screenshot: 'only-on-failure'. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Retries re-run a failed test. Screenshots/videos on failure give evidence. Retries are a safety net, not a fix.

3. Quarantine a flake with test.fixme or a bug ticket. Track retry rate in CI.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
retries: process.env.CI ? 2 : 0
use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

5. Three retries on a 10% flake still fail the nightly and waste 3x minutes.

451. What is video on failure?

1. Definition you should say first: use.video: 'retain-on-failure'. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Retries re-run a failed test. Screenshots/videos on failure give evidence. Retries are a safety net, not a fix.

3. Quarantine a flake with test.fixme or a bug ticket. Track retry rate in CI.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
retries: process.env.CI ? 2 : 0
use: { screenshot: 'only-on-failure', video: 'retain-on-failure' }
```

5. Three retries on a 10% flake still fail the nightly and waste 3x minutes.

452. What is a soft assertion?

1. Definition you should say first: expect.soft that does not stop the test immediately. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. expect.soft continues after failure so you collect many UI bugs in one run.

3. Use for long forms. End with a hard expect or the test may exit 0 incorrectly if you forget.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect.soft(page.getByLabel('City')).toHaveValue('Bengaluru');
await expect.soft(page.getByLabel('Pin')).toHaveValue('560001');
await expect(page.getByRole('button', { name: 'Save' })).toBeEnabled();
```

5. All-soft tests that never hard-fail hide broken setup.

453. What is test.describe?

1. Definition you should say first: A group of tests. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

454. What is test.beforeEach?

1. Definition you should say first: Runs before every test in the group. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

455. What is a tag in Playwright?

1. Definition you should say first: test.info().annotations or grep @smoke. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

456. How do you run only smoke?

1. Definition you should say first: npx playwright test --grep @smoke Then spend the rest of the answer on architecture, a working example, and a production failure.

2. describe groups tests. beforeEach runs setup. Tags via test() title @smoke or annotations let you grep.

3. Keep describe shallow. Put login in a fixture, not copy-paste beforeEach in 20 files.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test.describe('ATS form', () => {
  test.beforeEach(async ({ page }) => { await page.goto('/apply'); });
  test('@smoke fills notice', async ({ page }) => {
    await page.getByLabel('Notice').fill('30');
  });
});
# npx playwright test --grep @smoke
```

5. test.only left in a commit blocks the rest of CI if you forget forbidOnly.

457. What is a locator filter?

1. Definition you should say first: Narrow a locator by text or another locator. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. A locator is a lazy query that re-runs until the action succeeds. An ElementHandle is a snapshot that goes stale after rerender.

3. Create locators with getByRole / getByTestId / getByLabel. Chain filter and nth only after the role is correct.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const save = page.getByRole('button', { name: 'Save' });
await save.click();
await expect(save).toBeEnabled();
// bad: const h = await page.$('#save'); await h.click();
```

5. If you cache handles in a POM constructor, React rerenders will flake. Store locators, not handles.

458. What is getByLabel?

1. Definition you should say first: Find a control via its label text. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. getByLabel uses the associated label. getByPlaceholder is weaker because placeholders disappear when filled and are not always accessible.

3. Prefer label. If the control has no label, that is an a11y bug worth raising.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Current CTC (LPA)').fill('28');
await page.getByPlaceholder('dd/mm/yyyy').fill('03/09/2026');
```

5. Placeholder-only locators break when UX writes 'e.g. 30' instead of the old text.

459. What is getByPlaceholder?

1. Definition you should say first: Find an input by placeholder. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. getByLabel uses the associated label. getByPlaceholder is weaker because placeholders disappear when filled and are not always accessible.

3. Prefer label. If the control has no label, that is an a11y bug worth raising.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Current CTC (LPA)').fill('28');
await page.getByPlaceholder('dd/mm/yyyy').fill('03/09/2026');
```

5. Placeholder-only locators break when UX writes 'e.g. 30' instead of the old text.

460. What is a frame locator?

1. Definition you should say first: Enter an iframe without driver.switchTo. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. frameLocator enters an iframe without driver.switchTo. Locators pierce open shadow roots by default.

3. Chain frameLocator for nested frames (SSO widgets). Closed shadow roots need a test id on the host.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const stripe = page.frameLocator('iframe[name=stripe]');
await stripe.getByPlaceholder('Card number').fill('4242424242424242');
await page.locator('custom-widget').getByRole('button', { name: 'Pay' }).click();
```

5. Wrong frame = strict timeout. Confirm the iframe selector in the trace snapshot.

461. How do you work with Shadow DOM?

1. Definition you should say first: Locators pierce open shadow roots. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. frameLocator enters an iframe without driver.switchTo. Locators pierce open shadow roots by default.

3. Chain frameLocator for nested frames (SSO widgets). Closed shadow roots need a test id on the host.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const stripe = page.frameLocator('iframe[name=stripe]');
await stripe.getByPlaceholder('Card number').fill('4242424242424242');
await page.locator('custom-widget').getByRole('button', { name: 'Pay' }).click();
```

5. Wrong frame = strict timeout. Confirm the iframe selector in the trace snapshot.

462. What is fill vs type?

1. Definition you should say first: fill sets the value; type sends key events. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. fill sets the value and fires input events. type/pressSequentially sends keys. Use the latter when the app listens to each key (OTP, masks).

3. Clear then fill for React controlled inputs if fill alone does not stick.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Phone').fill('9876543210');
await page.getByLabel('OTP').pressSequentially('123456', { delay: 50 });
```

5. type into a filled field appends and fails validation. fill replaces.

463. When do you use pressSequentially?

1. Definition you should say first: When the app listens to each key. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. fill sets the value and fires input events. type/pressSequentially sends keys. Use the latter when the app listens to each key (OTP, masks).

3. Clear then fill for React controlled inputs if fill alone does not stick.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Phone').fill('9876543210');
await page.getByLabel('OTP').pressSequentially('123456', { delay: 50 });
```

5. type into a filled field appends and fails validation. fill replaces.

464. What is click force?

1. Definition you should say first: Bypasses some checks; use rarely. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. force: true clicks even if hidden. hover opens menus that need pointer over.

3. Prefer waiting for the overlay to hide. Use hover then click the menuitem role.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Profile' }).hover();
await page.getByRole('menuitem', { name: 'Logout' }).click();
// last resort: await locator.click({ force: true });
```

5. Force-clicking a covered button submits nothing. The trace will show the intercepting node.

465. What is hover?

1. Definition you should say first: Move the mouse for menus. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. force: true clicks even if hidden. hover opens menus that need pointer over.

3. Prefer waiting for the overlay to hide. Use hover then click the menuitem role.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByRole('button', { name: 'Profile' }).hover();
await page.getByRole('menuitem', { name: 'Logout' }).click();
// last resort: await locator.click({ force: true });
```

5. Force-clicking a covered button submits nothing. The trace will show the intercepting node.

466. How do you select a dropdown?

1. Definition you should say first: selectOption on a select, or click options in a custom combo. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.

3. Always assert after mutation. Counting before the table fetch completes is a race.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

467. How do you check a checkbox?

1. Definition you should say first: locator.check(). Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.

3. Always assert after mutation. Counting before the table fetch completes is a race.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

468. How do you read text?

1. Definition you should say first: locator.innerText() or toHaveText. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.

3. Always assert after mutation. Counting before the table fetch completes is a race.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

469. How do you count rows?

1. Definition you should say first: locator.count() or toHaveCount. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Native <select> uses selectOption. Custom combos are listbox/option roles. check() for checkboxes. innerText/toHaveText for copy. count/toHaveCount for lists.

3. Always assert after mutation. Counting before the table fetch completes is a race.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Country').selectOption({ label: 'India' });
await page.getByRole('checkbox', { name: 'Remote' }).check();
await expect(page.getByRole('row')).toHaveCount(11);
await expect(page.getByTestId('total')).toHaveText('INR 30 LPA');
```

5. Custom dropdowns that are divs will ignore selectOption. Use roles.

470. What is a clock in Playwright?

1. Definition you should say first: Fake timers for time-dependent UI. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. The clock API fakes timers, dates, and animations so expiry banners and OTPs are deterministic.
3. Install clock before the page uses Date.now. Run only the timers you need.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.clock.install({ time: new Date('2026-09-03T10:00:00') });
await page.goto('/offer');
await page.clock.fastForward('01:00:00');
await expect(page.getByText('Offer expired')).toBeVisible();
```

5. Faking time while a real setTimeout from analytics is running can hang. Pause and inspect.

471. What is authentication setup project?

1. Definition you should say first: A project that logs in once and writes storageState. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. storageState is a JSON dump of cookies and localStorage. A setup project logs in once and writes it.
3. Mark setup as a dependency of chromium/firefox projects. Never login with UI in every test unless you are testing login itself.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
setupProject: { name: 'setup', testMatch: /auth\.setup\.ts/ }
await page.context().storageState({ path: 'playwright/.auth/user.json' });
use: { storageState: 'playwright/.auth/user.json' }
```

5. Do not commit real SSO cookies. Use a test IdP and gitignore the auth folder if it has secrets.

472. How do you test localization?

1. Definition you should say first: A project per locale. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Projects can set locale, timezone, and device descriptors. That emulates viewport and UA, not a physical phone OS.
3. Real device farms (BrowserStack, Appium) are a different stack. Say so honestly.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
{ name: 'pixel', use: { ...devices['Pixel 7'] } }
{ name: 'de-DE', use: { locale: 'de-DE', timezoneId: 'Europe/Berlin' } }
```

5. Touch gestures and Safari bugs on iOS are not fully covered by Pixel emulation.

473. How do you test a mobile viewport?

1. Definition you should say first: Device descriptor in a project. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Projects can set locale, timezone, and device descriptors. That emulates viewport and UA, not a physical phone OS.
3. Real device farms (BrowserStack, Appium) are a different stack. Say so honestly.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
{ name: 'pixel', use: { ...devices['Pixel 7'] } }
{ name: 'de-DE', use: { locale: 'de-DE', timezoneId: 'Europe/Berlin' } }
```

5. Touch gestures and Safari bugs on iOS are not fully covered by Pixel emulation.

474. Does Playwright drive a real phone?

1. Definition you should say first: It emulates viewport and UA; real devices need another stack. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Projects can set locale, timezone, and device descriptors. That emulates viewport and UA, not a physical phone OS.
3. Real device farms (BrowserStack, Appium) are a different stack. Say so honestly.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
{ name: 'pixel', use: { ...devices['Pixel 7'] } }
{ name: 'de-DE', use: { locale: 'de-DE', timezoneId: 'Europe/Berlin' } }
```

5. Touch gestures and Safari bugs on iOS are not fully covered by Pixel emulation.

475. What is Playwright for Python vs Node?

1. Definition you should say first: Same browser engine; this kit uses Python in places and Node is common in web teams. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. Playwright is Microsoft's open-source library for reliable end-to-end tests. One API covers Chromium, Firefox, and WebKit, with browsers installed via `npx playwright install`.
3. In interviews, contrast it with Selenium: auto-wait, traces, API mocking, and test fixtures are built in. Node/TS is the default at most web teams; Python uses the same browser engine.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
npm init playwright@latest
```

```
npx playwright install
npx playwright test --project=chromium
npx playwright show-report
```

5. If you only say 'it automates browsers' you fail the next question. Name locators, contexts, traces, and storageState.

476. How do you install browsers?

1. Definition you should say first: playwright install. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.

3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

477. What is a browserType.launch?

1. Definition you should say first: Lower-level API vs Playwright Test fixtures. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.

3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

478. When do you use persistent context?

1. Definition you should say first: To keep a profile, as JobKit does for LinkedIn session. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.

3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

479. What is CDP?

1. Definition you should say first: Chrome DevTools Protocol; Playwright can connect over it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Node Playwright Test is the common CI choice. Python uses sync/async APIs. install downloads browsers. launch is the library API. Persistent context keeps a profile (JobKit LinkedIn). CDP is Chrome DevTools Protocol.

3. JobKit uses a persistent Edge/Chrome profile for hunt, not for submitting Easy Apply.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
from playwright.sync_api import sync_playwright
with sync_playwright() as p:
    ctx = p.chromium.launch_persistent_context(user_data_dir, channel='msedge', headless=False)
    page = ctx.pages[0]
    page.goto('https://www.linkedin.com')
```

5. Do not mix Test fixtures and raw launch in the same spec without knowing who owns the browser lifetime.

480. Why not click Submit in JobKit?

1. Definition you should say first: Sites forbid bots; you submit yourself after fill. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. JobKit Fill never clicks Submit. Sites forbid unattended apply bots. You review and submit.

3. Tests that exercise a public apply form should still not fire production applications. Use a dummy tenant.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.getByLabel('Phone').fill(phone);
await expect(page.getByRole('button', { name: 'Submit' })).toBeEnabled();
// do not: await page.getByRole('button', { name: 'Submit' }).click();
```

5. If a recruiter asks you to 'fully automate Easy Apply', decline. It violates LinkedIn terms and this kit's rules.

481. What is a POM?

1. Definition you should say first: Page Object Model: locators and actions in a class. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.

3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

482. Playwright and POM?

1. Definition you should say first: Use fixtures plus small page objects; avoid huge base classes. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.

3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

483. What is a component locator?

1. Definition you should say first: A locator root for a widget, then getByRole inside it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.

3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

484. How do you avoid xpath soup?

1. Definition you should say first: Prefer role, label, and test id. Then spend the rest of the answer on architecture, a working

example, and a production failure.

2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.

3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

485. What is an accessible name?

1. Definition you should say first: The name screen readers use; getByRole uses it. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. POM wraps locators and actions. In Playwright keep page objects thin and return locators. Accessible name is what getByRole matches. XPath soup is brittle.

3. Prefer a component locator root: this.table = page.getByRole('table', { name: 'Jobs' }).

4. Working code / syntax (write this on Playwright (TypeScript)):

```
export class JobsPage {
  constructor(private page: Page) {}
  jobs = this.page.getByRole('table', { name: 'Open jobs' });
  async open(title: string) {
    await this.jobs.getByRole('link', { name: title }).click();
  }
}
```

5. God-object base pages with 200 getters are unmaintainable. Split by screen.

486. How do you test file download names?

1. Definition you should say first: download.suggestedFilename(). Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Wait for the download event, then save to a unique path. Assert suggestedFilename and file bytes if needed.

3. Enable acceptDownloads (default in Playwright Test). Clean the folder in CI.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Export PDF' }).click(),
]);
expect(download.suggestedFilename()).toMatch(/interview-tosca\.pdf/);
await download.saveAs('test-results/tosca.pdf');
```

5. A shared Downloads directory across workers overwrites files.

487. How do you wait for URL?

1. Definition you should say first: expect(page).toHaveURL. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

488. How do you wait for title?

1. Definition you should say first: expect(page).toHaveTitle. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for

silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

489. What is networkidle?

1. Definition you should say first: Wait until no connections; often too brittle, prefer response waits. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

490. What is domcontentloaded?

1. Definition you should say first: HTML parsed; assets may still load. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

491. What is load?

1. Definition you should say first: Window load event. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

492. Which wait is default for goto?

1. Definition you should say first: Usually load; you can pass waitUntil. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveURL and toHaveTitle retry. goto waits for load by default. domcontentloaded is earlier. networkidle waits for silence and is brittle on analytics-heavy apps.

3. Prefer waiting for a response or a heading over networkidle.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.goto('/dashboard', { waitUntil: 'domcontentloaded' });
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle(/Job apply/);
await expect(page.getByRole('heading', { name: 'Inbox' })).toBeVisible();
```

5. SPAs that keep a websocket open never reach networkidle.

493. How do you debug a test?

1. Definition you should say first: PWDEBUG=1 or --debug and Trace Viewer. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PWDEBUG=1 or --debug opens Inspector. --ui is watch mode with time travel. Traces work after CI failures.

3. Reproduce with --repeat-each=5 on a flake. Step through locators in Inspector.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test tests/apply.spec.ts:12 --debug
npx playwright test --ui
$env:PWDEBUG=1; npx playwright test
```

5. Debugging only on your laptop misses CI viewport and timezone issues. Match CI config locally.

494. What is UI mode?

1. Definition you should say first: playwright test --ui for watch and time travel. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. PWDEBUG=1 or --debug opens Inspector. --ui is watch mode with time travel. Traces work after CI failures.

3. Reproduce with --repeat-each=5 on a flake. Step through locators in Inspector.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
npx playwright test tests/apply.spec.ts:12 --debug
npx playwright test --ui
$env:PWDEBUG=1; npx playwright test
```

5. Debugging only on your laptop misses CI viewport and timezone issues. Match CI config locally.

495. How do you seed test data?

1. Definition you should say first: APIRequestContext in beforeEach or a factory. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. APIRequestContext is HTTP without a browser. Use it to seed, login, and assert APIs.

3. Create the applicant via API, then open UI to verify the list. Faster and less flake.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
test('list shows seeded job', async ({ request, page }) => {
  await request.post('/api/jobs', { data: { title: 'Playwright QA', ctc: 30 } });
  await page.goto('/jobs');
  await expect(page.getByRole('cell', { name: 'Playwright QA' })).toBeVisible();
});
```

5. Do not put production bearer tokens in repo. Use env and a test tenant.

496. How do you hide secrets in CI?

1. Definition you should say first: Env vars, not committed .env. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. Secrets live in CI variables, not in specs or committed .env.

3. Read process.env.QA_PASSWORD. Fail fast if missing. Mask in logs.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
const password = process.env.QA_PASSWORD;
if (!password) throw new Error('QA_PASSWORD missing');
await page.getByLabel('Password').fill(password);
```

5. Traces can capture typed passwords. Use the password field and avoid extra screenshots of the vault.

497. What is a visual snapshot?

1. Definition you should say first: Screenshot compare with toHaveScreenshot. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveScreenshot compares pixels. Mask clocks, ads, and avatars. Stabilize fonts and animations.

3. Commit snapshots per OS/browser project. Update with --update-snapshots after a real UI change.

4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('clock'), page.getByRole('img', { name: 'avatar' })],
  maxDiffPixelRatio: 0.02,
});
```

5. Unmasked GIFs and webfonts from a CDN will fail Linux vs Windows.

498. Why do visual tests flake?

1. Definition you should say first: Fonts, animation, and time; mask dynamic regions. Then spend the rest of the answer on architecture, a working example, and a production failure.

2. toHaveScreenshot compares pixels. Mask clocks, ads, and avatars. Stabilize fonts and animations.
3. Commit snapshots per OS/browser project. Update with --update-snapshots after a real UI change.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('clock'), page.getByRole('img', { name: 'avatar' })],
  maxDiffPixelRatio: 0.02,
});
```

5. Unmasked GIFs and webfonts from a CDN will fail Linux vs Windows.

499. What is a mask in screenshots?

1. Definition you should say first: Hide volatile locators before compare. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. toHaveScreenshot compares pixels. Mask clocks, ads, and avatars. Stabilize fonts and animations.
3. Commit snapshots per OS/browser project. Update with --update-snapshots after a real UI change.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await expect(page).toHaveScreenshot('dashboard.png', {
  mask: [page.getByTestId('clock'), page.getByRole('img', { name: 'avatar' })],
  maxDiffPixelRatio: 0.02,
});
```

5. Unmasked GIFs and webfonts from a CDN will fail Linux vs Windows.

500. How do you test accessibility?

1. Definition you should say first: axe via a Playwright plugin or extra library. Then spend the rest of the answer on architecture, a working example, and a production failure.
2. a11y: axe scans. console: fail on pageerror if the team agrees. basic auth: httpCredentials. geo: grantPermissions. ads: route abort. HAR: replay captured traffic.
3. Pick one extra check per suite so CI stays fast. HAR is great for flaky third parties.
4. Working code / syntax (write this on Playwright (TypeScript)):

```
await page.route(/ads\./, (r) => r.abort());
await context.grantPermissions(['geolocation']);
await context.setGeolocation({ latitude: 12.97, longitude: 77.59 });
await context.routeFromHAR('fixtures/jobs.har', { url: '**/api/jobs*' });
```

5. Failing on every console.warn will brick a suite that uses a noisy third-party widget.